

# 第一章

# 机器学习与优化范式

肖桐 杨曦涵 李天园 刘宏宇

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室

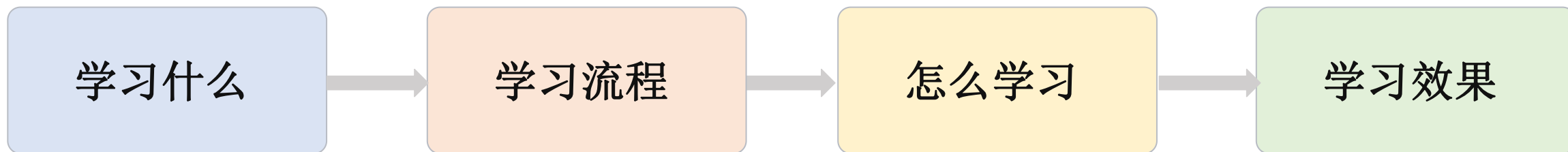


東北大學  
Northeastern University



# 1.1 机器学习与优化范式

- 1.1.1 监督学习、无监督学习和自监督学习
- 1.1.2 机器学习训练流程
- 1.1.3 优化算法与训练过程
- 1.1.4 模型评估与选择
- 1.1.5 正则化机制



# 为什么需要机器学习

- 为什么需要机器学习?
- 什么场景需要机器学习?
- 机器学习解决了什么问题?

# 为什么需要机器学习?

## 传统程序：人写规则

例子：判断成绩是否及格

```
if score >= 60:  
    result = “及格”  
else:  
    result = “不及格”
```

数据

+

人工规则



结果

- 规则从哪里来：人编写规则
- 缺点：只能判断规则明确的事情

## 缺点：很多场景无法总结规则

例子：垃圾邮件识别



正常邮件

您的论文已经被iclr27  
接收，请及时参会

正常



垃圾邮件

您抽中大奖，请明天来  
领取两袋鸡蛋

错误

- 规则不明确、无法描述

# 为什么需要机器学习

- 在很多场景下，并不能提取出一套固定的规则来处理输入输出

## 人工规则（示例）

- 出现“免费”“中奖”“赚钱” → 垃圾邮件
- 包含多个链接或附件 → 垃圾邮件
- 过多感叹号 → 垃圾邮件



正常邮件

中奖用户信息已经确定，  
开会对一下



垃圾  
邮件



垃圾邮件

请明天来领取安慰奖两  
袋鸡蛋



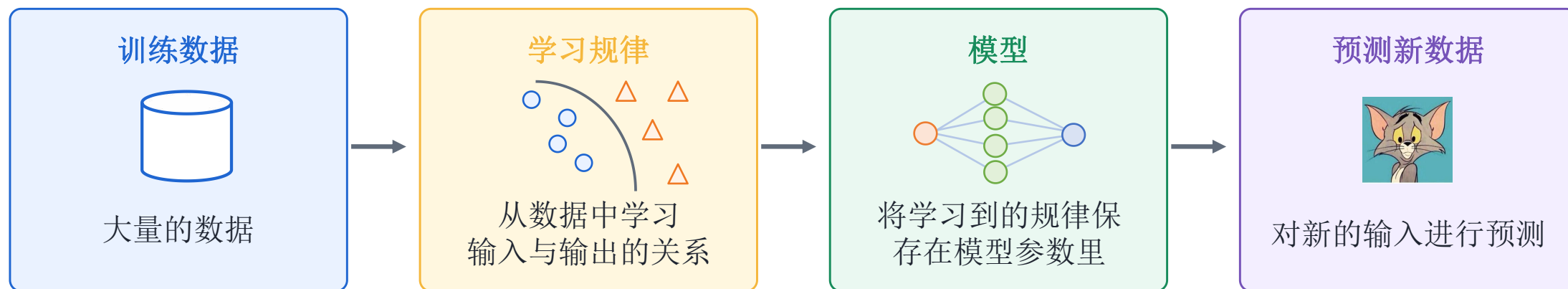
正常  
邮件

**需要一套非常复杂的规则，能够适应多种输入**

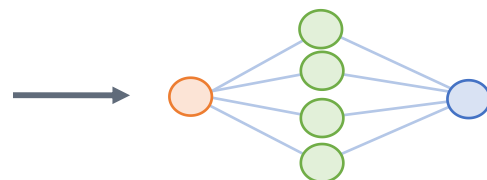
因此，我们引入机器学习，来捕捉复杂的规律模式

# 什么是机器学习

机器学习是一种让计算机从数据中自动学习规律，并利用这些规律进行预测、分类或决策的方法。本章节将对这几个过程进行详细讲解



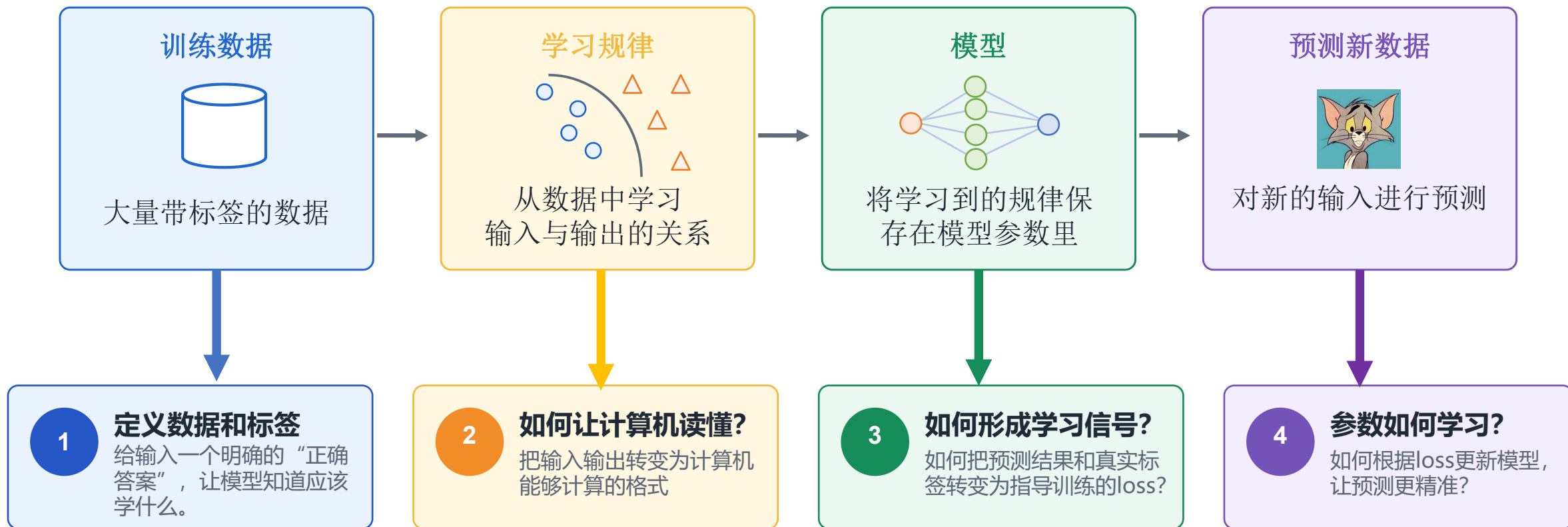
## 例子：猫狗识别



新图片

预测结果： 猫

因此，为了能够实现这个过程，我们需要解决一些问题，也就是本章节的学习目标



# 机器学习在学习什么？

1

## 定义数据和标签

给输入一个明确的“正确答案”，让模型知道应该学什么。

2

## 如何让计算机读懂？

把输入输出转变为计算机能够计算的格式

3

## 如何形成学习信号？

如何把预测结果和真实标签转变为指导训练的loss？

4

## 参数如何学习？

如何根据loss更新模型，让预测更精准？

- 如何给输入的数据一个正确的答案，这个正确的答案就是标签/监督信号
- **机器学习，就是在学习输入和标签的映射关系**
- 对于标签的不同类别，我们可以分为监督学习、无监督学习和自监督学习。
- 根据建模目标不同，可以分为生成式建模和判别式建模

# 1.1 监督学习

- 把输入数据和对应的标签称为监督信号。

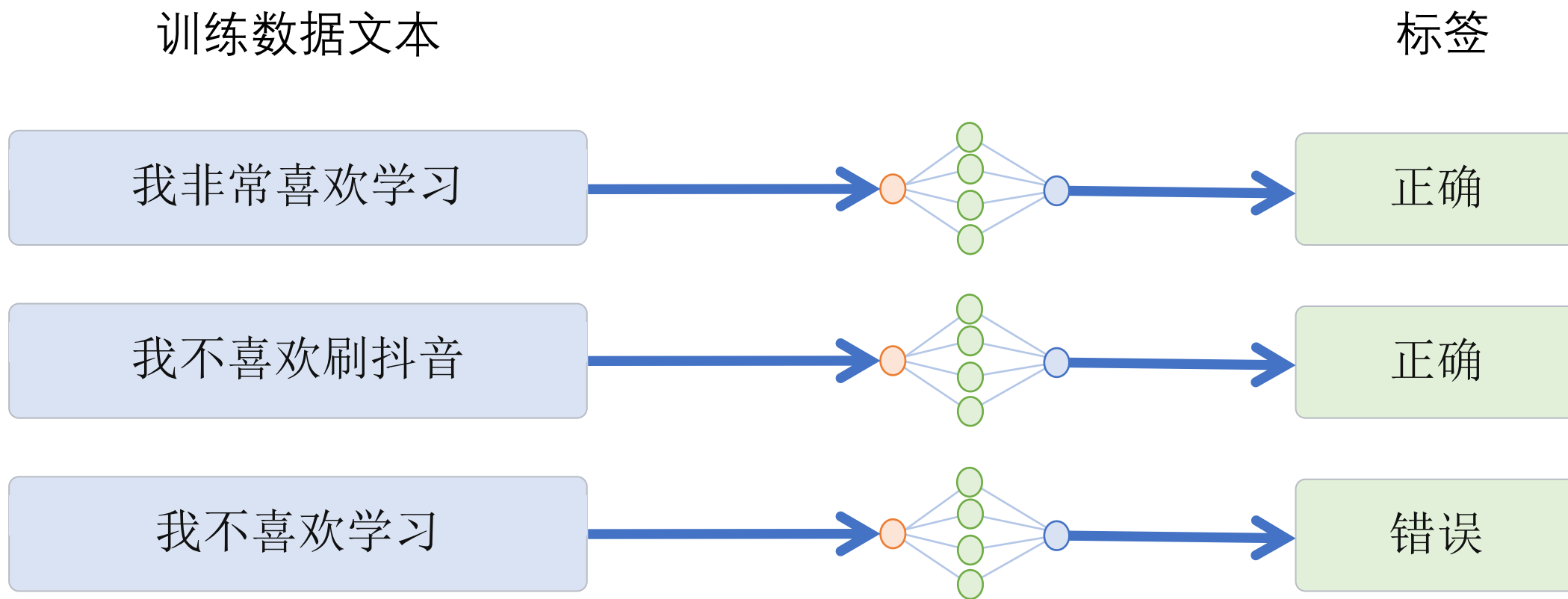


- 对于标签的不同类别，我们可以分为监督学习、无监督学习和自监督学习。

一般标签为人为设定，用来告诉模型我们希望你的学习什么

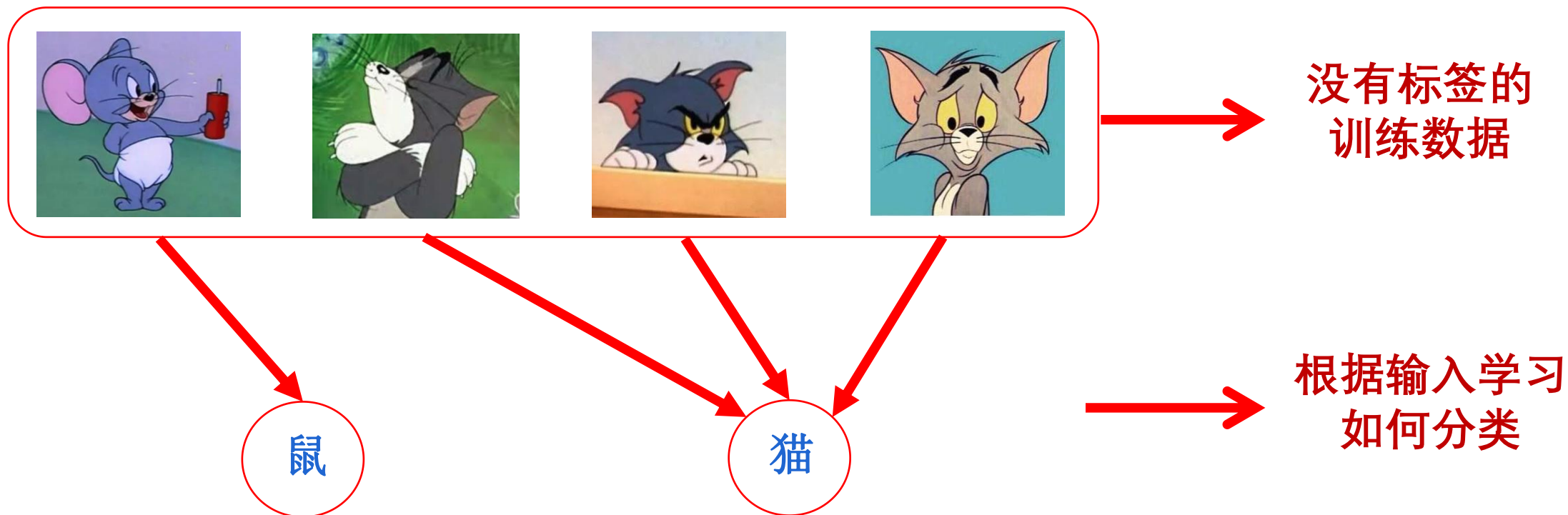
# 1.1 有监督学习

有监督学习就是给对应训练数据，用人工来对数据进行打标签。往往标签质量对学习效果有很大影响。



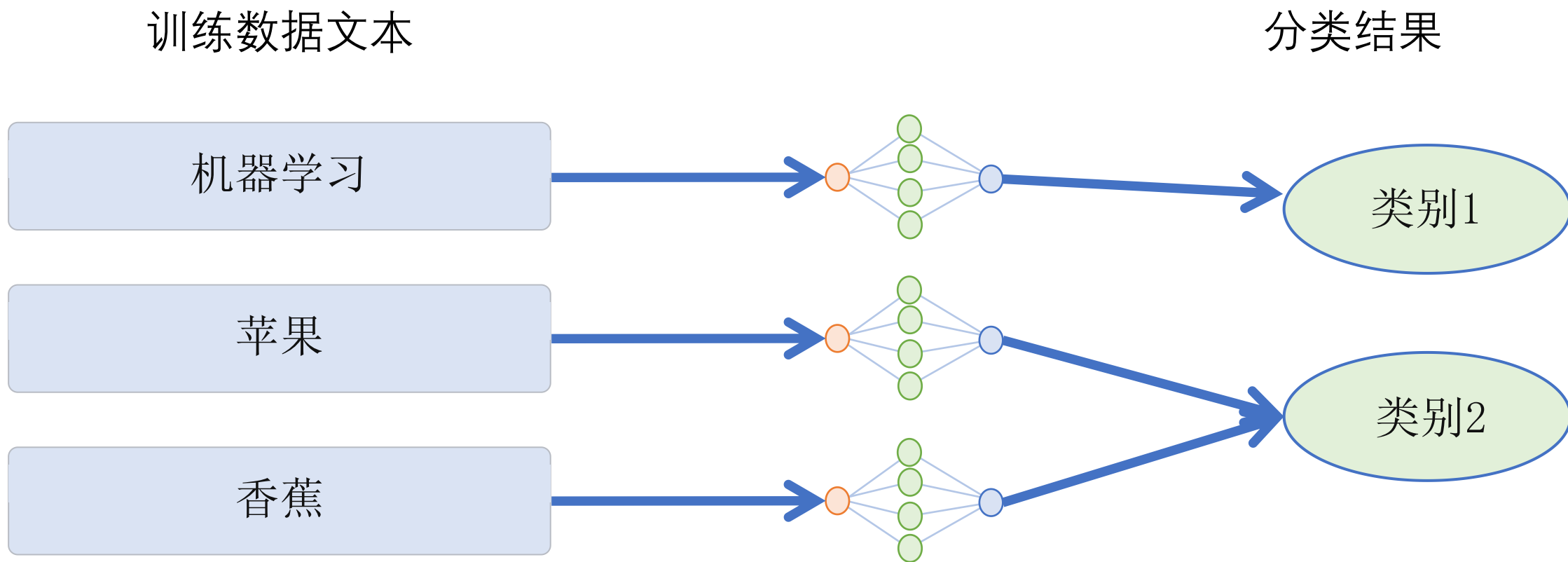
# 1.1 无监督学习

训练数据没有人工标注的标签，模型自己根据数据的分布去对数据做分类



# 1.1 无监督学习

训练数据没有人工标注的标签，模型自己根据数据的分布去对数据做分类



# 1.1 自监督学习

答案本来就藏在原始数据里，我们只需要把它“变成训练目标”，例如对原始数据随机掩码，然后将原始文本作为学习目标。这样不需要额外的监督信号。

原始训练数据文本

我非常喜欢学习

我不喜欢刷抖音

训练数据随机加掩码

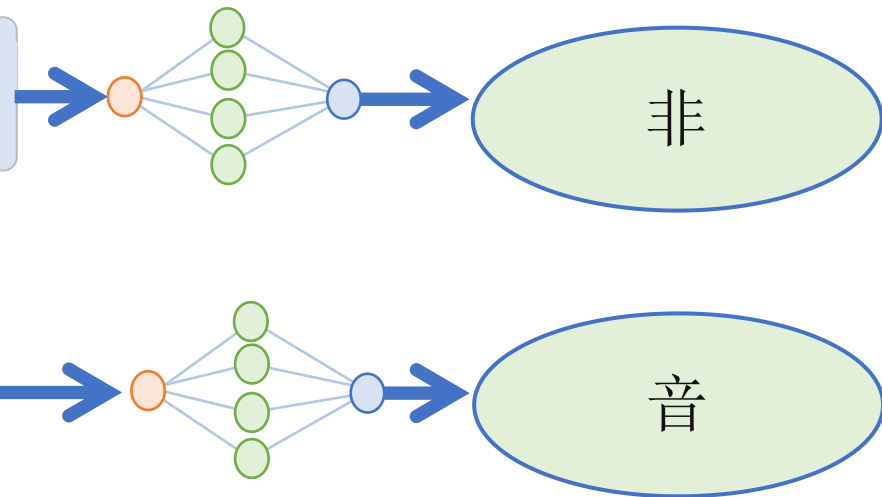
我 [mask] 常喜欢学习

我不喜欢刷抖 [mask]

预测掩码内容

非

音



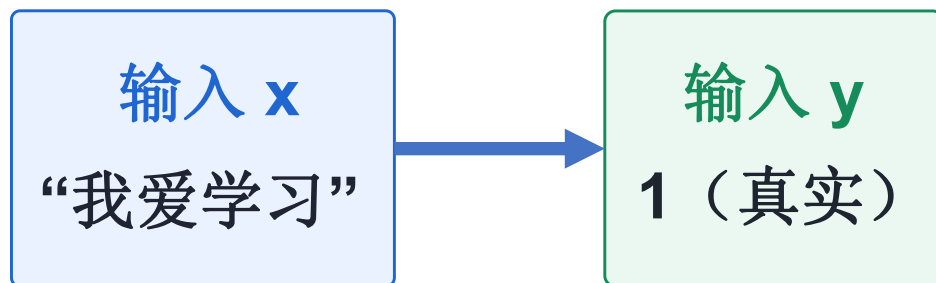
# 1.1 判别式机器学习

- 按照学习内容的不同，可以分为生成式模型和判别式模型

## 判别式

给定输入的 $x$ ，对 $x$ 的类别进行判断

情感分类任务预测情感真实性

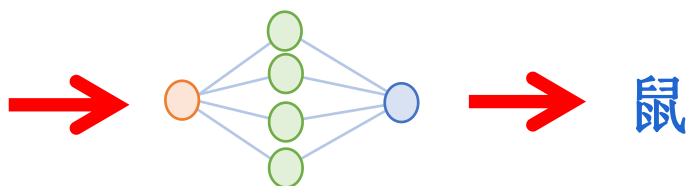


判别式是将学习一套规则，这套规则可以学习输入与类别标签的规则，从而实现类别判断。

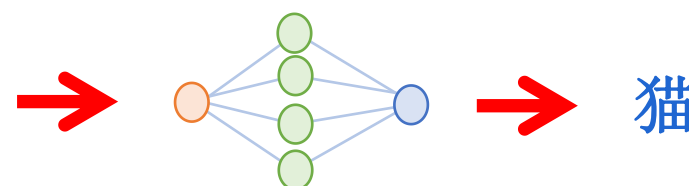
判别式建模更像是对输入信息进行“任务相关的压缩”：保留完成判断所需要的信息，丢弃与分类无关的细节。

# 1.1 判别式机器学习

- 常见判别式任务
- 图像分类：只需要判断图片是猫还是鼠



鼠



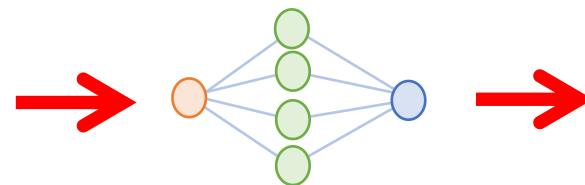
猫

- 垃圾邮件检测：输入邮件，输出是否是垃圾邮件



正常邮件

今天有机器学习考试

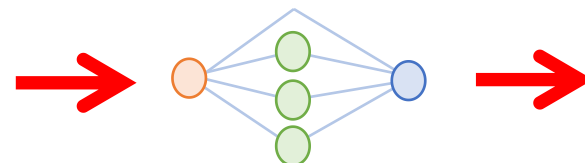


正常邮件



垃圾邮件

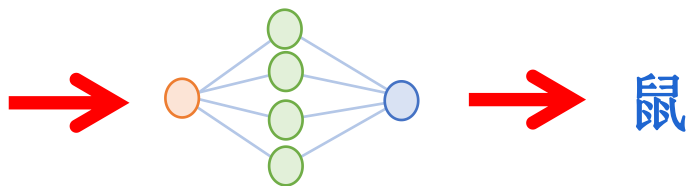
您的冰红茶中了再来一瓶



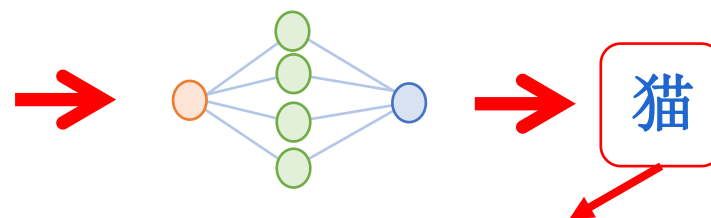
垃圾邮件

# 1.1 判别式机器学习

- 常见判别式任务
- 图像分类：只需要判断图片是猫还是鼠



鼠



猫

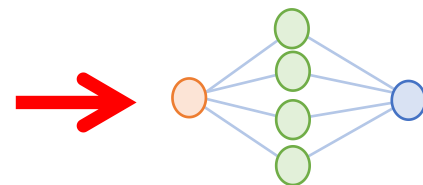
一般是向量、1/0的标签值

- 垃圾邮件检测：输入邮件，输出是否是垃圾邮件



正常邮件

今天有机器学习考试

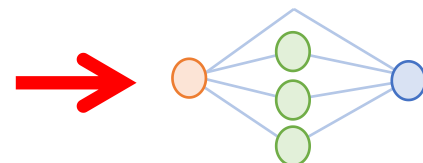


正常邮件



垃圾邮件

您的冰红茶中了再来一瓶



垃圾邮件

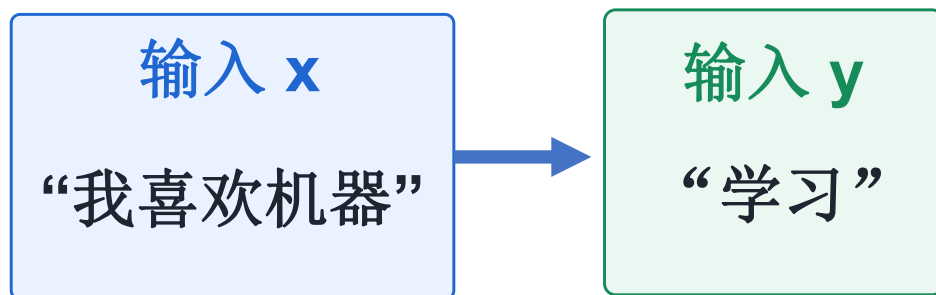
# 1.1 生成式机器学习

- 1.2.2 判别式建模与生成式

## 生成式

给定输入的 $x$ ，生成后续内容

文本生成、翻译任务



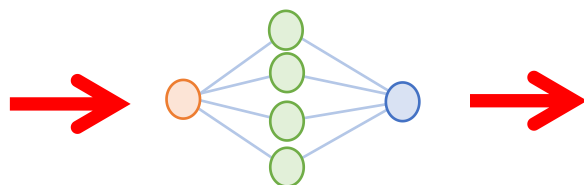
生成式模型：学习数据本身的分布和规律，并根据这些规律生成新的数据

判别式关注“怎么判断”，生成式关注“数据是怎么产生的，怎么产生新的数据”

# 1.1 生成式机器学习

- 常见生成式任务
- 图像生成：给出一段文本，生成对应图片

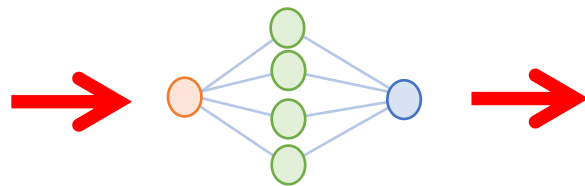
生成一张猫的图片



- 机器翻译任务：给出原文，进行翻译

翻译原文

我非常喜欢高级机器学习课程



翻译结果

I really like the Advanced Machine Learning course.

# 1.1 输入向量化

1

## 定义数据和标签

给输入一个明确的“正确答案”，让模型知道应该学什么。

2

## 如何让计算机读懂？

把输入输出转变为计算机能够计算的格式

3

## 如何形成学习信号？

如何把预测结果和真实标签转变为指导训练的loss？

4

## 参数如何学习？

如何根据loss更新模型，让预测更精准？

- 对于输入，如何变成能让计算机读懂并且能计算的数据格式？
- 我们通常把数据转化向量作为计算单元
- 这里我们介绍one hot编码和词袋向量两种方法

# 1.1词袋向量

词袋向量将一段句子表示为固定词表中各词出现次数组成的向量

词表  $V = \{ \text{我, 喜欢, 机器, 学习, 人工智能, 模型, 数据, 深度, 猫, 电影} \}$

| 句子           | 我 | 喜欢 | 机器 | 学习 | 人工智能 | 模型 | 数据 | 深度 | 猫 | 电影 | 词袋向量                    |
|--------------|---|----|----|----|------|----|----|----|---|----|-------------------------|
| 我 喜欢 机器 学习   | 1 | 1  | 1  | 1  | 0    | 0  | 0  | 0  | 0 | 0  | [1,1,1,1,0,0,0,0,0,0,0] |
| 我 喜欢 人工智能 模型 | 1 | 1  | 0  | 0  | 1    | 1  | 0  | 0  | 0 | 0  | [1,1,0,0,1,1,0,0,0,0,0] |

这样，我们把这两个句子表示为了两个向量。

# 1.1词袋向量

词袋向量将一段句子表示为固定词表中各词出现次数组成的向量

词表  $V = \{ \text{我, 喜欢, 机器, 学习, 人工智能, 模型, 数据, 深度, 猫, 电影} \}$

| 句子           | 我 | 喜欢 | 机器 | 学习 | 人工智能 | 模型 | 数据 | 深度 | 猫 | 电影 | 词袋向量                    |
|--------------|---|----|----|----|------|----|----|----|---|----|-------------------------|
| 我 喜欢 机器 学习   | 1 | 1  | 1  | 1  | 0    | 0  | 0  | 0  | 0 | 0  | [1,1,1,1,0,0,0,0,0,0,0] |
| 我 喜欢 人工智能 模型 | 1 | 1  | 0  | 0  | 1    | 1  | 0  | 0  | 0 | 0  | [1,1,0,0,1,1,0,0,0,0,0] |

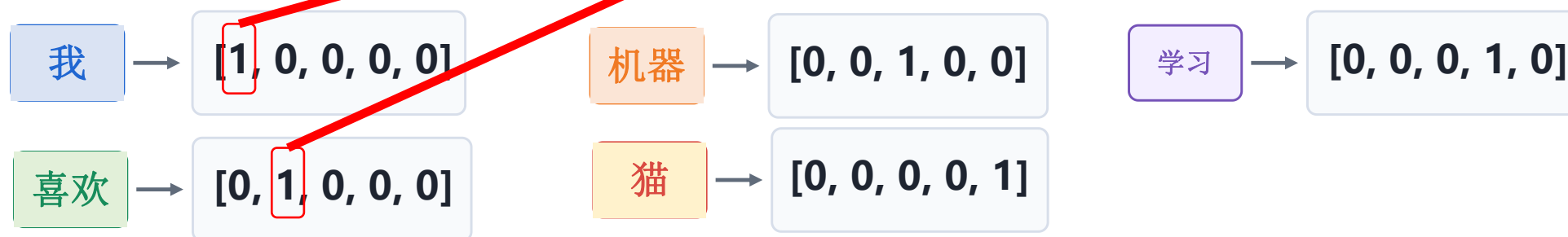
这样，我们把这两个句子表示为了两个向量。

# 1.1 one hot编码

- one hot编码可以对词表中的每个词进行编码

词表中只有 5 个词

$V = \{\text{我, 喜欢, 机器, 学习, 猫}\}$



**One-Hot:** 向量维度 = 词表大小; 只有当前词对应的位置为 1, 其余位置全部为 0。

文本例子

“我 喜欢 机器 学习”



## 1.1.2 机器如何预测

1

### 定义数据和标签

给输入一个明确的“正确答案”，让模型知道应该学什么。

2

### 如何让计算机读懂？

把输入输出转变为计算机能够计算的格式

3

### 如何形成学习信号？

如何把预测结果和真实标签转变为指导训练的loss？

4

### 参数如何学习？

如何根据loss更新模型，让预测更精准？

- 完成输入输出的表示，如何让计算机进行学习？
- 用什么方式才能让机器进行学习？

## 1.1.2 如何进行完整前向计算

- 现在我们已经把输入表示为了向量，如何让机器学习到输入和标签之间的关系？ ---神经网络
- 神经网络可以看作一个可学习的函数： $\mathbf{x} \xrightarrow{\text{神经网络 } f_{\theta}} \hat{\mathbf{y}}$
- 神经网络本质上就是一组可以学习的矩阵和向量，我们用一个最简单的线性层举例

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

输入向量

$$x = [0 \quad 0 \quad 1 \quad 0 \quad 0]$$

## 1.1.2如何进行完整前向计算

- 现在我们已经把输入表示为了向量，如何让机器学习到输入和标签之间的关系？ ---神经网络
- 神经网络可以看作一个可学习的函数： $\mathbf{x} \xrightarrow{\text{神经网络 } f_{\theta}} \hat{\mathbf{y}}$
- 神经网络本质上就是一组可以学习的矩阵和向量

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

↓  
输出向量

## 1.1.2如何进行完整前向计算

- 现在我们已经把输入表示为了向量，如何让机器学习到输入和标签之间的关系？ ---神经网络
- 神经网络可以看作一个可学习的函数： $\mathbf{x} \xrightarrow{\text{神经网络 } f_{\theta}} \hat{\mathbf{y}}$
- 神经网络本质上就是一组可以学习的矩阵和向量

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

↓  
激活函数

## 1.1.2 如何进行完整前向计算

- 现在我们已经把输入表示为了向量，如何让机器学习到输入和标签之间的关系？ ---神经网络

- 神经网络可以看作一个可学习的函数： $\mathbf{x} \xrightarrow{\text{神经网络 } f_{\theta}} \hat{\mathbf{y}}$

- 神经网络本质上就是一组可以学习的矩阵和向量

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

可学习的参数

$$W = \begin{bmatrix} 0.8 & 0.1 \\ 0.7 & 0.2 \\ 0.2 & 1.5 \\ 0.6 & 0.3 \\ 0.9 & 0.1 \end{bmatrix}$$

$$\mathbf{b} = [0.1 \quad -0.1]$$

## 1.1.2如何进行完整前向计算

- 因此，一次机器学习的预测过程，可以视作一次矩阵与矩阵加法的结合

$$y = xW + b$$

$$xW = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0.8 & 0.1 \\ 0.7 & 0.2 \\ 0.2 & 1.5 \\ 0.6 & 0.3 \\ 0.9 & 0.1 \end{bmatrix} = \begin{bmatrix} 0.2 & 1.5 \end{bmatrix}$$

$$y = \begin{bmatrix} 0.2 & 1.5 \end{bmatrix} + \begin{bmatrix} 0.1 & -0.1 \end{bmatrix} = \begin{bmatrix} 0.3 & 1.4 \end{bmatrix}$$

## 1.1.2如何进行完整前向计算

- 激活函数用于对神经网络每一层的计算结果进行变换，使模型能够学习更复杂的输入与输出关系。
- 激活函数决定神经元在得到计算结果后，应该如何继续输出，并为神经网络引入非线性能力。



什么是非线性能力？为什么需要非线性能力？

## 1.1.2如何进行完整前向计算

- 假设神经网络有两层：

$$h = xW_1 + b_1$$

$$y = hW_2 + b_2$$

- 把第一层代入第二层后，可以通过矩阵变换，将两个参数矩阵合并：

$$y = (xW_1 + b_1)W_2 + b_2 \longrightarrow y = xW + b$$

- 如果没有激活函数，无论叠多少层，整个神经网络本质上仍然只是一次线性变换。

## 1.1.2如何进行完整前向计算

- 假设神经网络有两层：

$$h = xW_1 + b_1$$

$$y = hW_2 + b_2$$

- 把第一层代入第二层后，可以通过矩阵变换，将两个参数矩阵合并：

$$y = (xW_1 + b_1)W_2 + b_2 \longrightarrow y = xW + b$$

- 如果没有激活函数，无论叠多少层，整个神经网络本质上仍然只是一次线性变换。
- 激活函数通常是非线性的，不能通过矩阵乘法合并。多层网络就不能再简单合并成一个矩阵，从而可以学习更复杂的关系。

## 1.1.2如何进行完整前向计算

- 在这里，用softmax作为激活函数举例：

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

↓  
激活函数

$$\text{Softmax}(y_i) = \frac{e^{y_i}}{\sum_{j=1}^2 e^{y_j}}$$

$$p_1 = \frac{e^{0.3}}{e^{0.3} + e^{1.4}} \approx 0.25 \longrightarrow \text{类别1的概率}$$

$$p_2 = \frac{e^{1.4}}{e^{0.3} + e^{1.4}} \approx 0.75 \longrightarrow \text{类别2的概率}$$

## 1.1.2损失函数

- 损失函数

设模型输出为  $y_\theta = f_\theta(x)$  标准答案为  $y_i$  单个样本的损失  $\mathcal{L}(y_\theta, y_{\text{gold}})$  来度量模型输出和标准答案之间的差异。根据输入和输出的不同，我们以两种损失函数举例：

平均绝对误差MAE: 
$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

均方误差MSE: 
$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

# 1.1.2损失函数

- MAE损失函数

直接计算“预测值和真实值相差多少”。

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

更关注大误差

误差越大，惩罚增长越快

例子：预测一个数值

样本1

真实值  $y = 3$

预测值  $\hat{y} = 5$

$$MAE = |3-5| = 2$$

样本2

真实值  $y = 4$

预测值  $\hat{y} = 1$

$$MAE = |1-4| = 3$$

$$\text{误差大小} = (2+3) / 2 = 2.5$$

# 1.1.2损失函数

- MSE损失函数

把误差平方后再求平均，因此会更重地惩罚“大误差”。

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

更关注大误差

误差越大，惩罚增长越快

例子：预测一个数值

样本1

真实值  $y = 3$

预测值  $\hat{y} = 5$

$$MSE = (3-5)^2 = 4$$

样本2

真实值  $y = 4$

预测值  $\hat{y} = 1$

$$MSE = (1-4)^2 = 9$$

$$\text{误差大小} = (4+9) / 2 = 6.5$$

## 1.1.2损失函数

- 上述的例子，我们得到了一层线性层的输出 $(p1, p2) = (0.25, 0.75)$ ，在这个二分类任务中，真实标签为 $(0, 1)$ ，接下来我们可以计算损失函数，来衡量当前预测值和真实值的差距。

$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b}) = [0.25 \quad 0.75]$$

$$\text{MAE} = \frac{1}{2} (|0.25 - 0| + |0.75 - 1|)$$

$$\text{MSE} = \frac{1}{2} [(0.25 - 0)^2 + (0.75 - 1)^2]$$

# 1.1.3有了预测结果如何进行学习

1

## 定义数据和标签

给输入一个明确的“正确答案”，让模型知道应该学什么。

2

## 如何让计算机读懂？

把输入输出转变为计算机能够计算的格式

3

## 如何形成学习信号？

如何把预测结果和真实标签转变为指导训练的loss？

4

## 参数如何学习？

如何根据loss更新模型，让预测更精准？

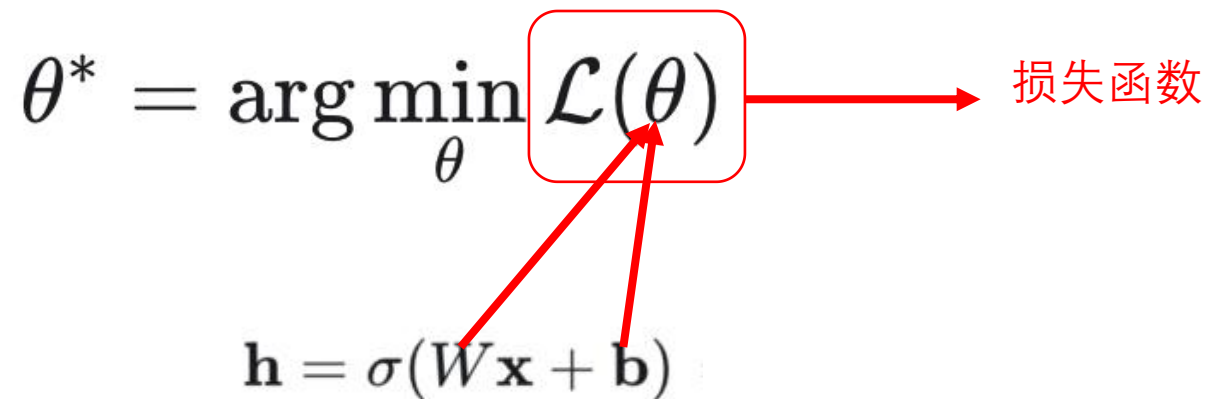
- 在得到预测值和真实值的差距之后，如何让参数矩阵更新，使输入更加接近输出？

# 1.1.3反向传播与梯度下降

- 知道和真实值的差距之后，如何更新两个可学习的参数？--优化问题
- 假设当前的可学习的参数为  $\theta$ ，那么优化问题变成了找到新的参数  $\theta^*$ ，让损失函数最小。也就是：

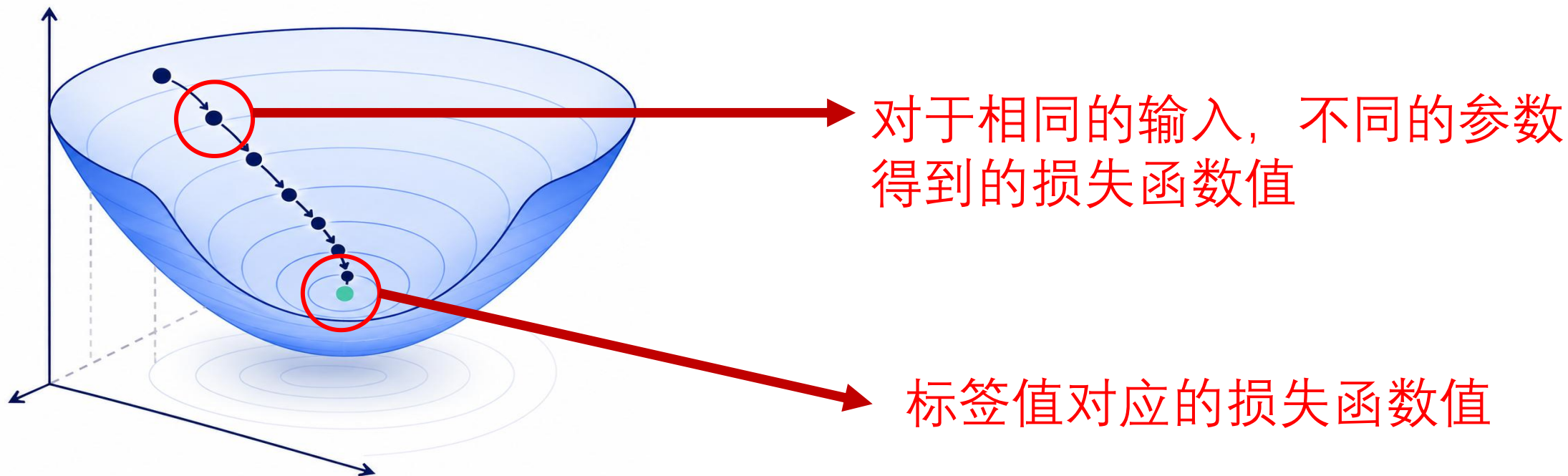
$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

损失函数


$$\mathbf{h} = \sigma(W\mathbf{x} + \mathbf{b})$$

# 1.1.3反向传播与梯度下降

- 更形象的来说，所有可学习参数共同构成一个参数空间，而损失函数会给参数空间中的每一个位置对应一个损失值。训练的目标，就是不断调整参数，让当前位置的损失尽可能降低。我们需要找到一个参数移动的方向，使损失函数下降得尽可能快。



# 1.1.3反向传播与梯度下降

- 更形象的来说，所有可学习参数共同构成一个参数空间，而损失函数会给参数空间中的每一个位置对应一个损失值。训练的目标，就是不断调整参数，让当前位置的损失尽可能降低。我们需要找到一个参数移动的方向，使损失函数下降得尽可能快。

梯度

$$\nabla_{\theta} \mathcal{L}(\theta)$$

# 1.1.3反向传播与梯度下降

- 更形象的来说，所有可学习参数共同构成一个参数空间，而损失函数会给参数空间中的每一个位置对应一个损失值。训练的目标，就是不断调整参数，让当前位置的损失尽可能降低。我们需要找到一个参数移动的方向，使损失函数下降得尽可能快。

梯度  $\nabla_{\theta} \mathcal{L}(\theta)$

- 什么是梯度？梯度是函数在某一点“增长最快的方向”。

对于函数  $y = f(x_1, x_2, \dots, x_n)$

其梯度为  $\nabla f = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$

# 1.1.3反向传播与梯度下降

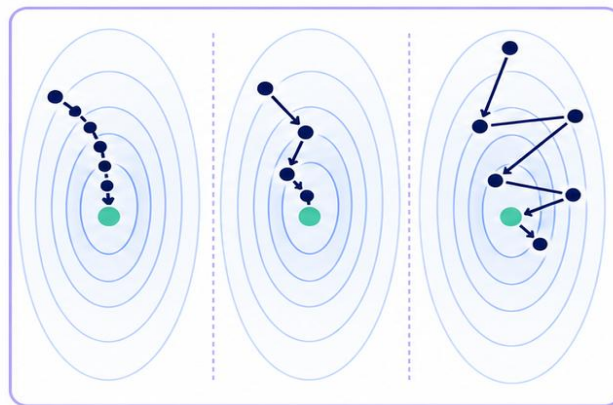
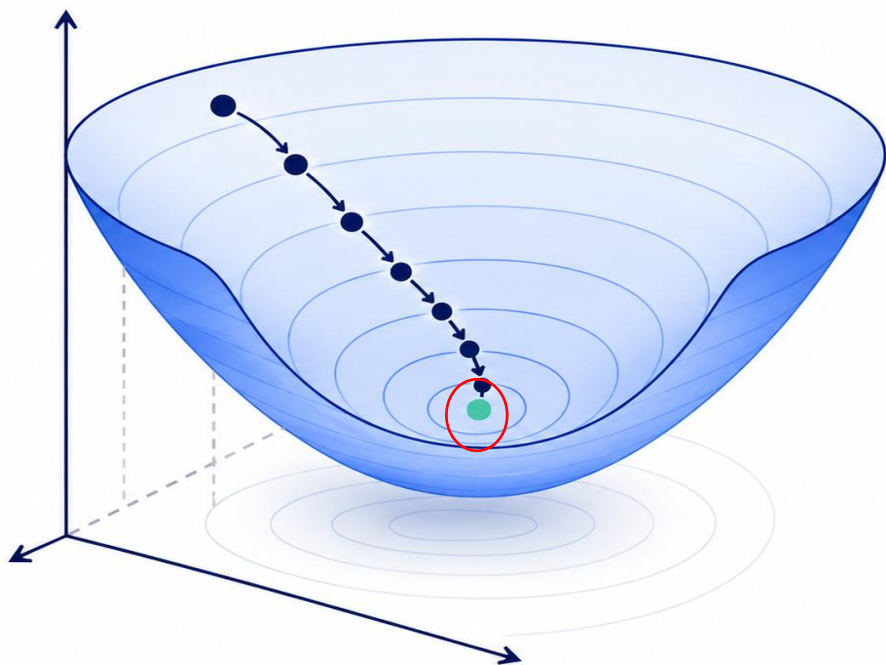
- 更形象的来说，所有可学习参数共同构成一个参数空间，而损失函数会给参数空间中的每一个位置对应一个损失值。训练的目标，就是不断调整参数，让当前位置的损失尽可能降低。我们需要找到一个参数移动的方向，使损失函数下降得尽可能快。

梯度  $\nabla_{\theta} \mathcal{L}(\theta)$

- 梯度给出了函数增长最快的方向，因此梯度的反方向就是函数下降最快的方向。

# 1.1.3反向传播与梯度下降

- 梯度给出了函数增长最快的方向，因此梯度的反方向就是函数下降最快的方向。这样，让参数沿着梯度的负方向移动，便能最快到达损失函数最低点。

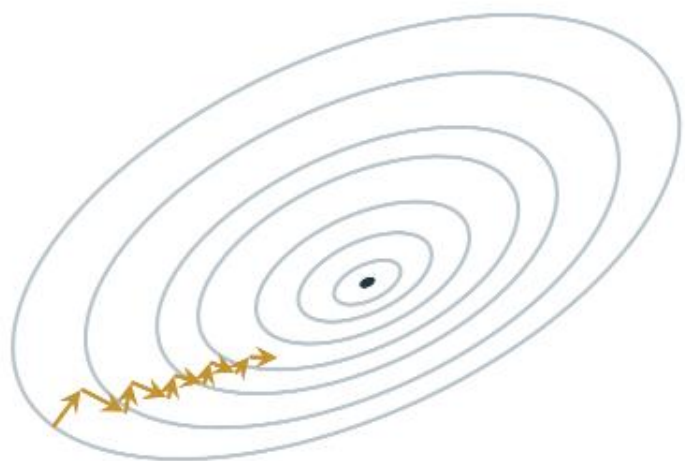


# 1.1.3反向传播与梯度下降

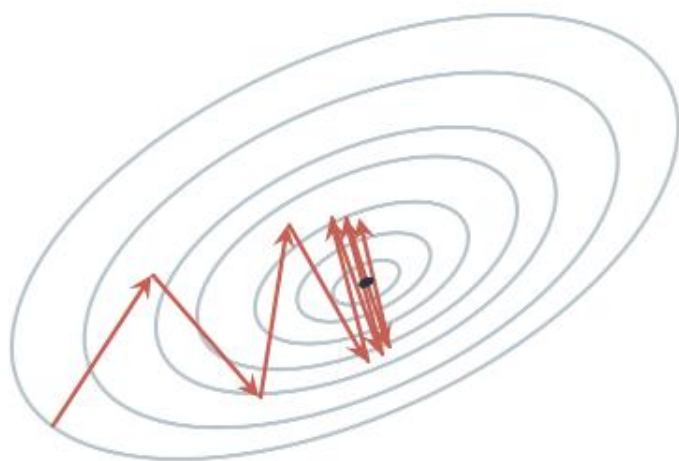
- 然而，沿着梯度方向一次要走多长也是一个需要确定的问题，走的太短计算成本高，走的步长太大会引起震荡。因此，当前机器学习的梯度下降通常用学习率来控制。梯度下降的公式为：

$$\theta_{t+1} = \theta_t - \boxed{\eta} \nabla_{\theta} \mathcal{L}(\theta_t)$$

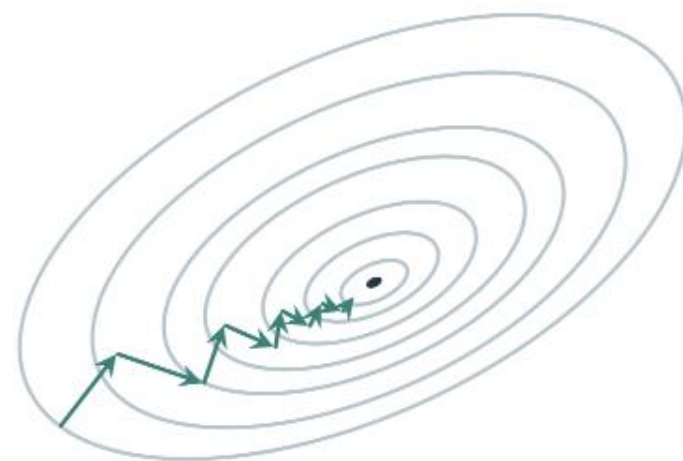
学习率



(a) 学习率过小



(b) 学习率过大



(c) 学习率合适

# 1.1.3反向传播与梯度下降

- 进行实际的计算之前，让我们明确一个问题，我们计算的梯度，**是谁对谁的梯度？**
- 要确定这个问题，我们要回到我们的优化目标上

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

# 1.1.3反向传播与梯度下降

- 再进行实际的计算之前，让我们明确一个问题，我们计算的梯度，**是谁对谁的梯度？**
- 要确定这个问题，我们要回到我们的优化目标上

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

- **我们的优化目标是，求出一组参数，让损失函数最小。**
- **因此要求出损失函数对参数矩阵的梯度**

# 1.1.3反向传播与梯度下降

- 要计算梯度，首先要计算导数
- 然而，对于损失函数  $\mathcal{L}$  与参数矩阵  $W$ ，直接算会非常复杂，因为损失函数是权重矩阵的复杂复合函数。因此在计算的时候用链式法则

## 1.1.3 反向传播与梯度下降

- 要计算梯度，首先要计算导数
- 然而，对于损失函数  $\mathcal{L}$  与参数矩阵  $W$ ，直接算会非常复杂，因为损失函数是权重矩阵的复杂复合函数。因此在计算的时候用链式法则

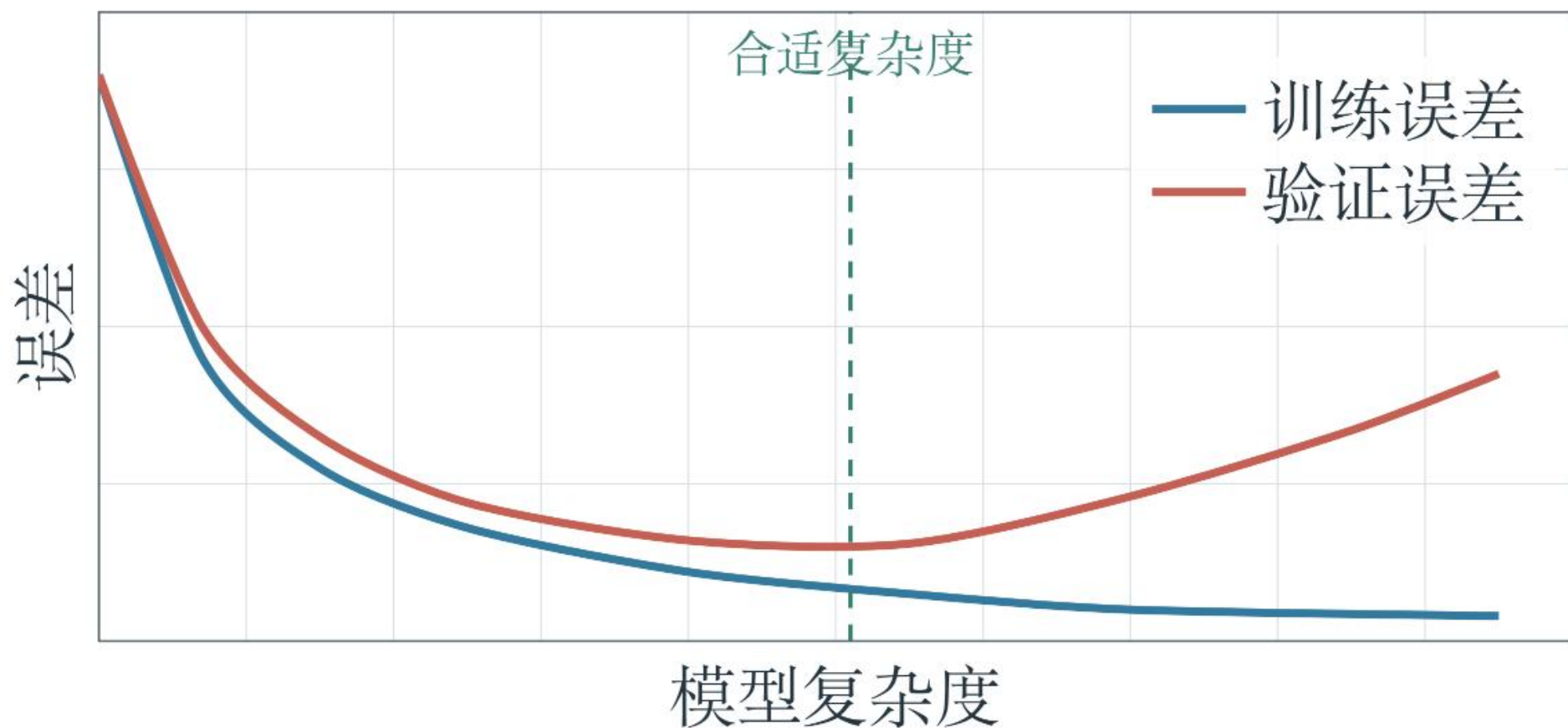
例子：对于单层的线性网络  $\hat{y} = wx + b$      $\mathcal{L} = |\hat{y} - y|$

我们可以得到链式法则为  $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w}$

但通常网络都比较复杂，因此反向传播的计算开销也会越来越大

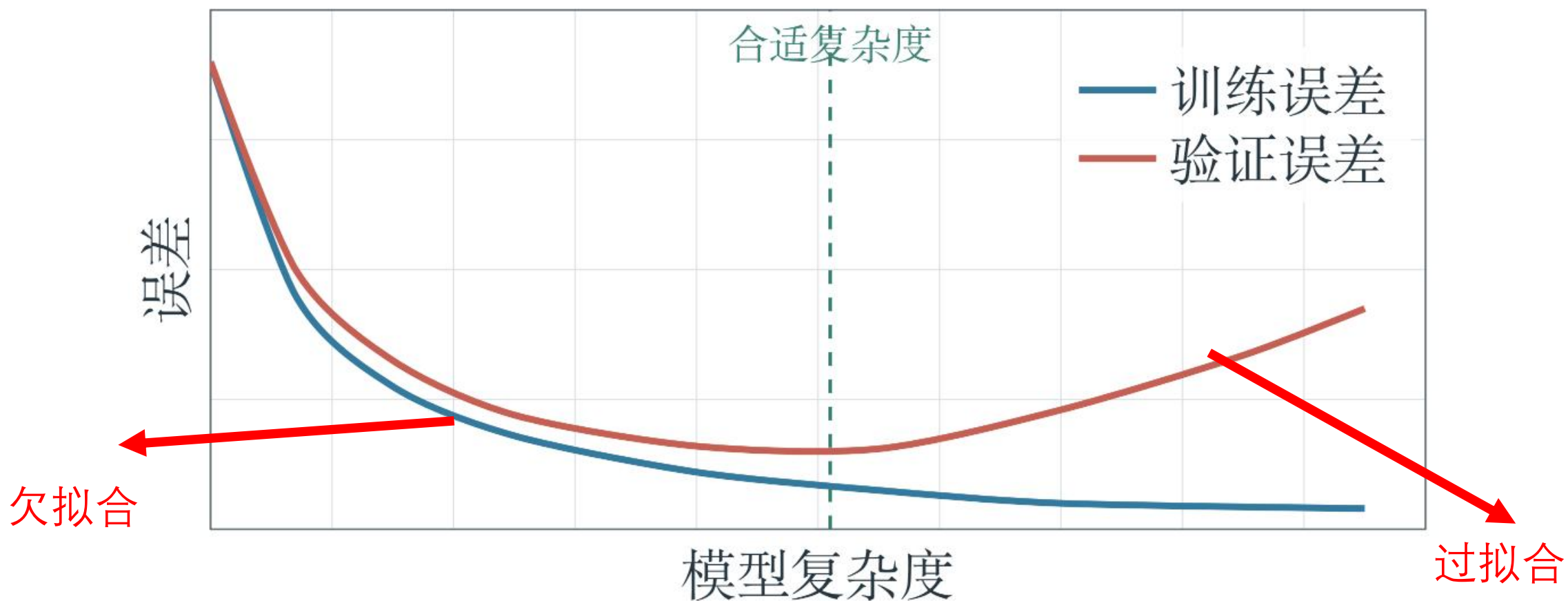
## 1.1.4选择最优参数

- 但并不是越训练越好。训练过多会造成过拟合问题，在训练集上表现最佳，在测试集上反而性能快速下降



## 1.1.4选择最优参数

- 但并不是越训练越好。训练过多会造成过拟合问题，在训练集上表现最佳，在测试集上反而性能快速下降

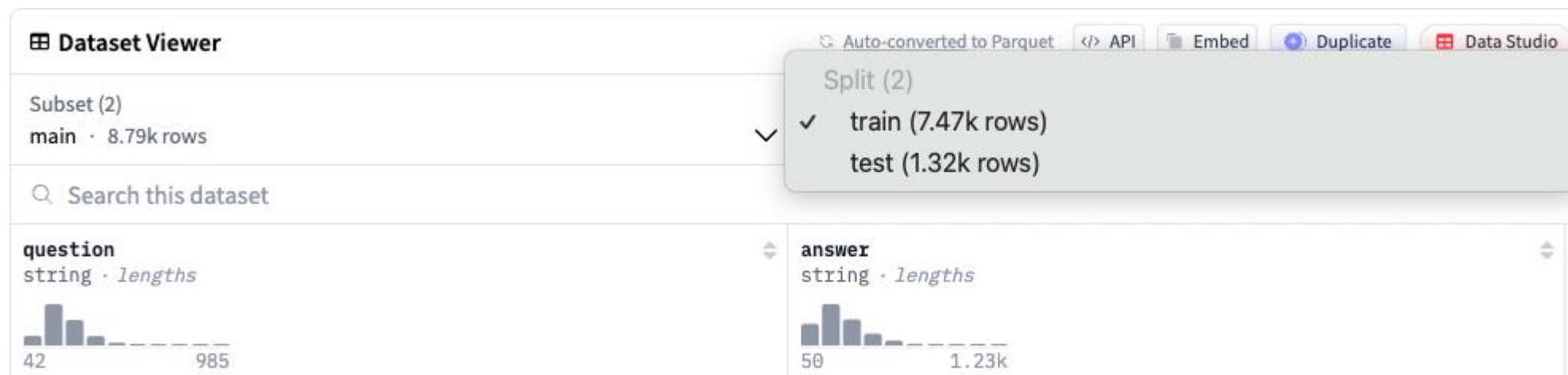


## 1.1.4选择最优参数

- 我们通常将所有数据划分为三部分：训练集、验证集、测试集
- 训练集进行数据训练，验证集挑选最佳参数，测试集是最终的测评任务

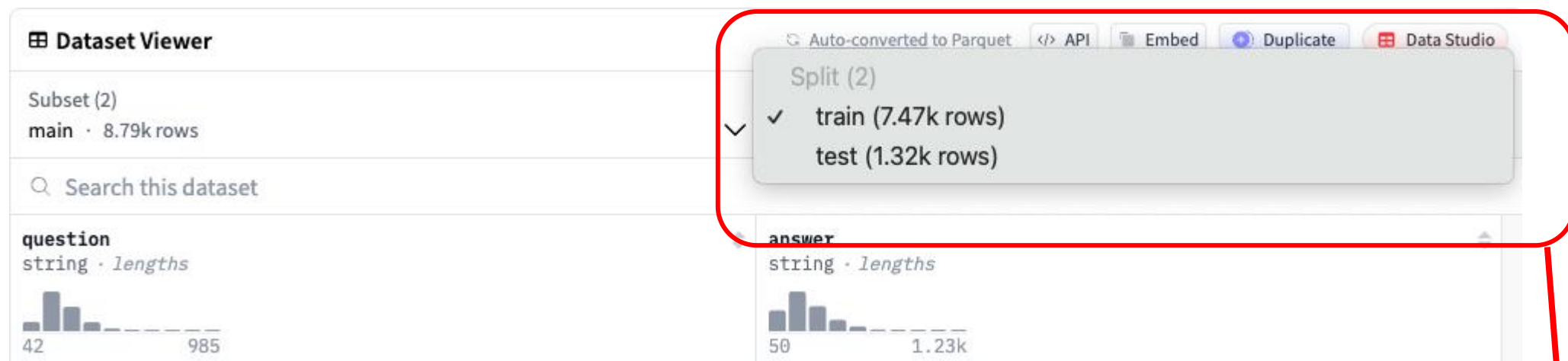
## 1.1.4选择最优参数

- 我们通常将所有数据划分为三部分：训练集、验证集、测试集
- 训练集进行数据训练，验证集挑选最佳参数，测试集是最终的测评任务



## 1.1.4选择最优参数

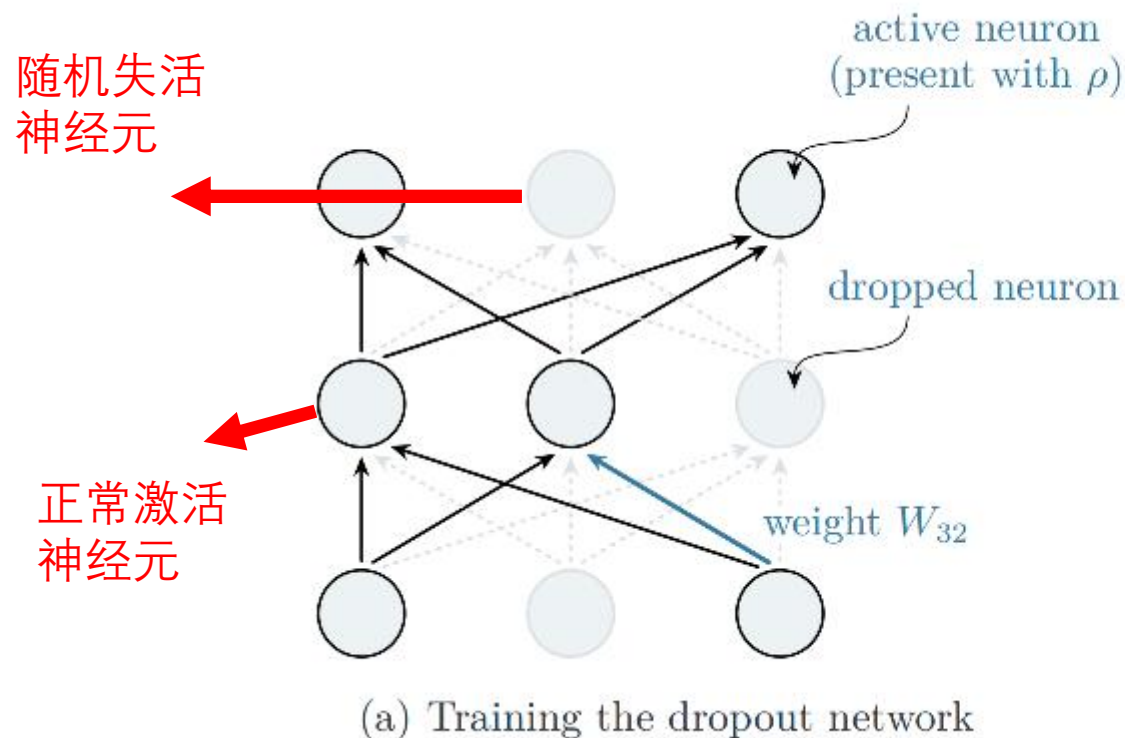
- 我们通常将所有数据划分为三部分：训练集、验证集、测试集
- 训练集进行数据训练，验证集挑选最佳参数，测试集是最终的测评任务



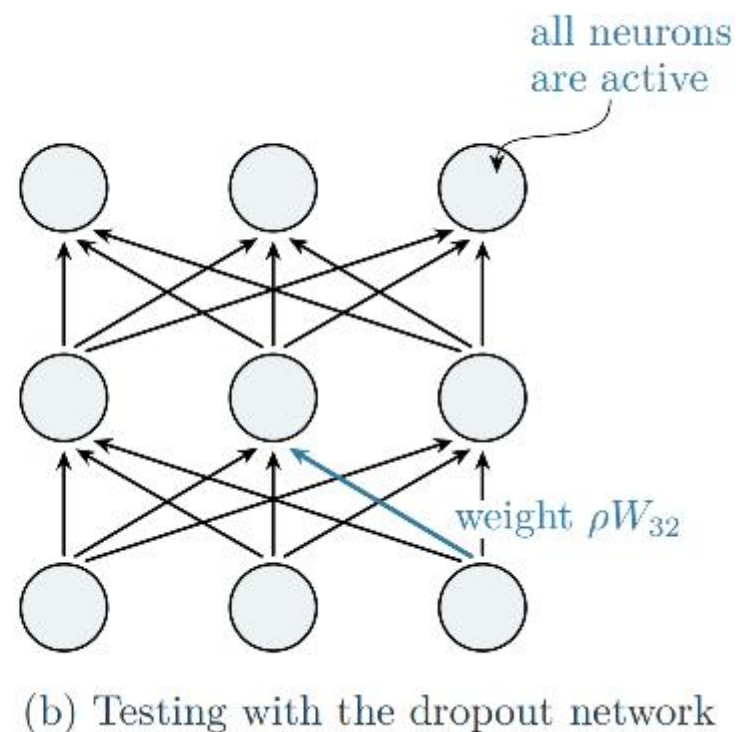
有些任务已经划分好了训练集  
测试集

## 1.1.5 正则化

- 正则化是控制模型复杂度、缓解过拟合的重要方法。
- dropout 是神经网络中非常常用的正则化方法。它在训练时随机屏蔽部分神经元连接，使网络不能过度依赖某些特定隐藏单元



训练时随机失活神经元



训练完成测试时激活全部神经元

# 第一章

## 第1.2讲数学基础

肖桐 杨曦涵 李天园 刘宏宇

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室



東北大學  
Northeastern University



## 1.2 数学基础

- 在这里用一个前向传播反向传播的完整计算过程的例子，来了解整个过程和数学基础

## 1.2完整计算过程

- 这里进行一次完整的计算过程

还是词性分类的例子，网络用单一线性层  $z = xW + b$

使用sigmoid激活函数  $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$x = [0 \quad 0 \quad 1 \quad 0 \quad 0] \quad W = \begin{bmatrix} -0.4 \\ 0.1 \\ 1.2 \\ 0.8 \\ -1.0 \end{bmatrix} \quad b = -0.2$$

## 1.2完整计算过程

- 先计算前向得到输出  $z = xW + b$

$$z = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -0.4 \\ 0.1 \\ 1.2 \\ 0.8 \\ -1.0 \end{bmatrix} - 0.2 \longrightarrow z = 1.0$$

## 1.2完整计算过程

- 将输出的向量经过sigmoid激活函数后，得到预测值

$$\hat{y} = \sigma(z) \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = \frac{1}{1 + e^{-1}} \approx 0.731$$

## 1.2完整计算过程

- 二分类使用交叉熵损失函数

$$\mathcal{L} = - [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- 带入后, 得到  $\mathcal{L} \approx 0.313$
- 至此, 一次前向过程已经全部算完, 接下来算反向传播的过程

## 1.2 计算过程

- 完整链式法则: 
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W}$$

$$\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$\hat{y} = \sigma(z) \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad z = xW + b$$

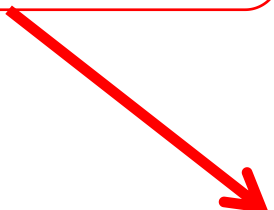
- 先算三项偏导数，然后再相乘

## 1.2完整计算过程

- 完整链式法则:  $\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W}$

$$\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$


$$\frac{\partial}{\partial \hat{y}} (-y \log \hat{y}) = -\frac{y}{\hat{y}}$$


$$\frac{\partial}{\partial \hat{y}} [-(1 - y) \log(1 - \hat{y})] = \frac{1 - y}{1 - \hat{y}}$$

## 1.2完整计算过程

• 完整链式法则:  $\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W}$

$$\mathcal{L} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

$$\frac{\partial}{\partial \hat{y}} (-y \log \hat{y}) = -\frac{y}{\hat{y}}$$

$$\frac{\partial}{\partial \hat{y}} [-(1 - y) \log(1 - \hat{y})] = \frac{1 - y}{1 - \hat{y}}$$

相加通分

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

## 1.2完整计算过程

- 完整链式法则: 
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \boxed{\frac{\partial \hat{y}}{\partial z}} \frac{\partial z}{\partial W}$$

- 根据  $\hat{y} = \sigma(z)$  
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- 可以得到 
$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

## 1.2完整计算过程

- 完整链式法则：
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W}$$

- 前两项的偏导数相乘之后可以化简为

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} = \hat{y} - y$$

## 1.2完整计算过程

- 完整链式法则：
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial W}$$

- 前两项的偏导数可以化简为

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y}) \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} = \hat{y} - y$$

- 将  $\hat{y} = 0.731$  及  $y = 1$  代入可以得到 
$$\frac{\partial \mathcal{L}}{\partial z} = -0.269$$

## 1.2完整计算过程

- 完整链式法则：
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \boxed{\frac{\partial z}{\partial W}}$$

- 最终，我们可以得到最后的梯度为，这个的计算结果，就是权重矩阵参数更新的大小

$$\frac{\partial \mathcal{L}}{\partial W} = x^T (\hat{y} - y) \quad \frac{\partial \mathcal{L}}{\partial W} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} (-0.269)$$

## 1.2完整计算过程

- 学习率0.1, 根据公式  $W' = W - \eta \frac{\partial \mathcal{L}}{\partial W}$  我们可以算出新的矩阵参数为:

$$W = \begin{bmatrix} -0.4 \\ 0.1 \\ 1.2 \\ 0.8 \\ -1.0 \end{bmatrix} \longrightarrow W' = \begin{bmatrix} -0.4 \\ 0.1 \\ 1.2269 \\ 0.8 \\ -1.0 \end{bmatrix}$$

- 同理可以算出新的偏执值为:

$$b' = -0.1731$$

## 1.2完整计算过程

- 用相同的输入，再次做一下前向，我们可以得到

$$z' = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} -0.4 \\ 0.1 \\ 1.2269 \\ 0.8 \\ -1.0 \end{bmatrix} - 0.1731 = 1.0538$$

- 更接近1的标签值  $\hat{y}' = \sigma(1.0538) \approx 0.7415$
- 经过这样重复的计算，不断拉近输出与标签的距离，就完成了机器学习的过程

# 第一章

## 第1.3讲深度神经网络基础

肖桐 杨曦涵 李天园 刘宏宇

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室



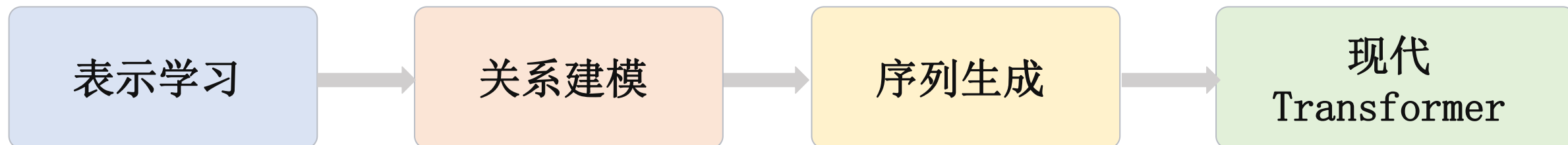
東北大學  
Northeastern University



# 1.3 深度神经网络基础

- 多层感知机如何学习连续表示
- 注意力机制如何建立跨位置关联
- Encoder/Decoder 如何组织输入到输出的生成

从“变换表示”走向“动态检索”，再走向“序列生成”

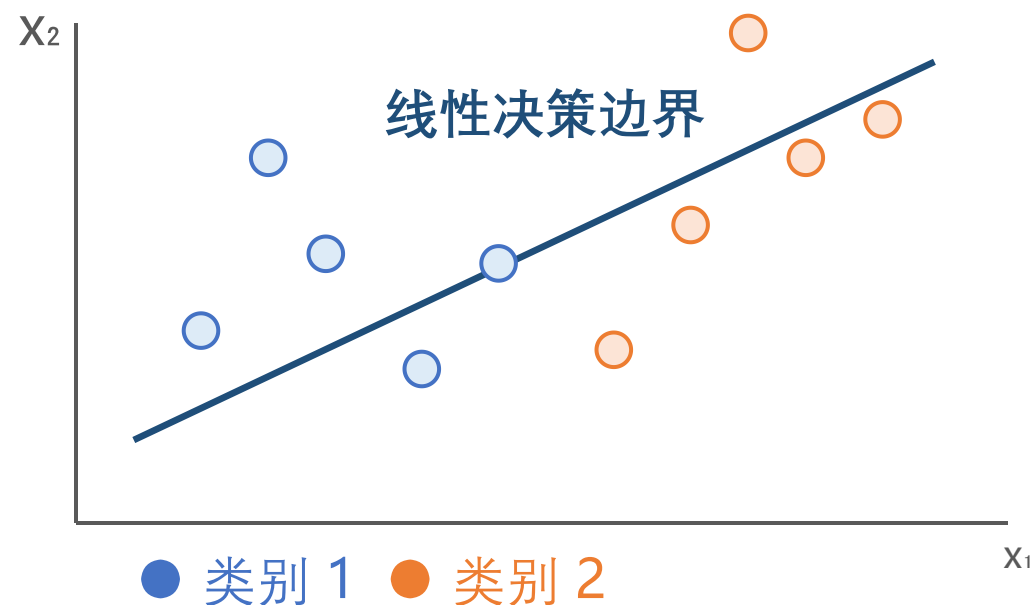


# 1.3 深度神经网络基础

## 单层感知机：从平面二分类开始

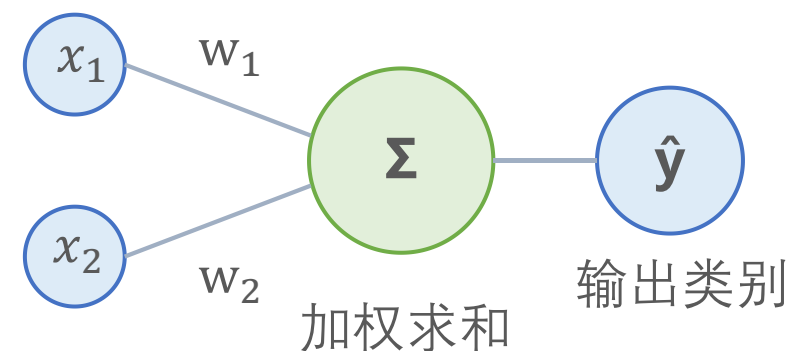
为什么需要单层感知机？——先看一个最简单的分类问题：在二维平面中，将两类样本分开。

平面二分类



学习一条  
决策边界

单层感知机



$f(x) = w^T x + b \rightarrow \text{激活函数} \rightarrow \text{分类}$

核心：用一个神经元学习线性决策边界，把二维输入映射为二分类结果。

# 1.3 深度神经网络基础

- 单层感知机

单个感知机首先对输入进行仿射变换：

$$y = w^T x + b$$

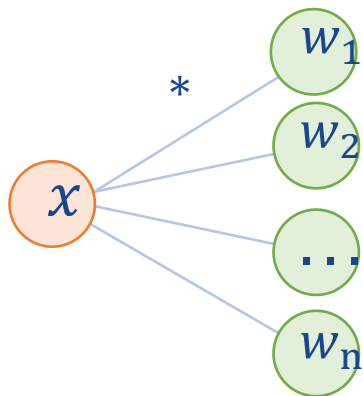
$x$

# 1.3 深度神经网络基础

- 单层感知机

单个感知机首先对输入进行仿射变换：

$$y = w^T x + b$$

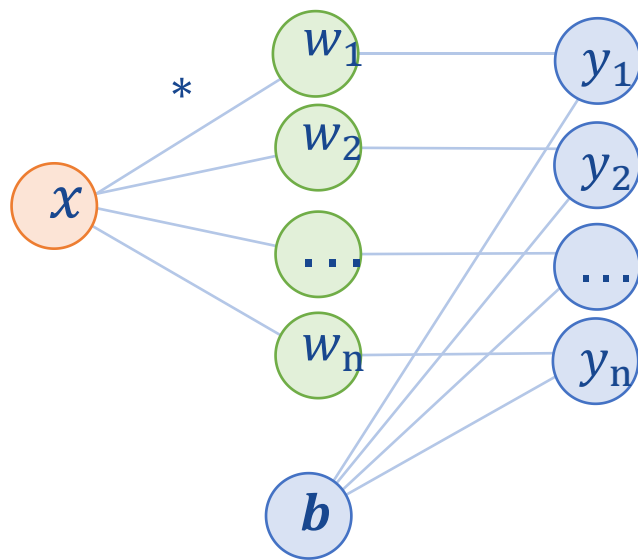


# 1.3 深度神经网络基础

- 单层感知机

单个感知机首先对输入进行仿射变换：

$$y = w^T x + b$$

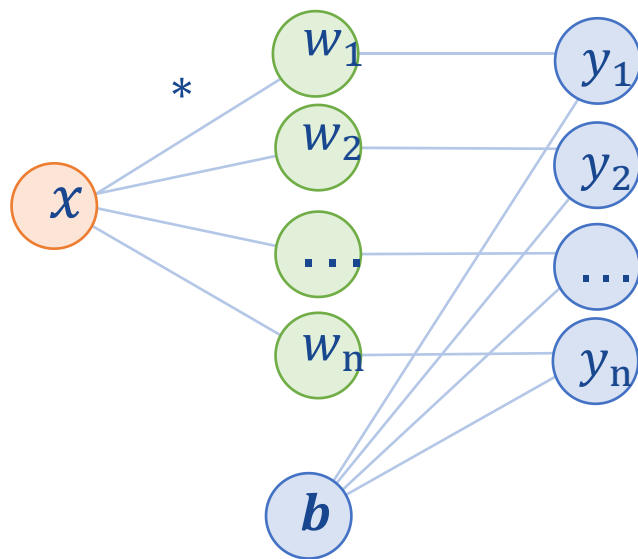


# 1.3 神经网络基础

- 单层感知机

单个感知机首先对输入进行仿射变换：

$$y = w^T x + b$$



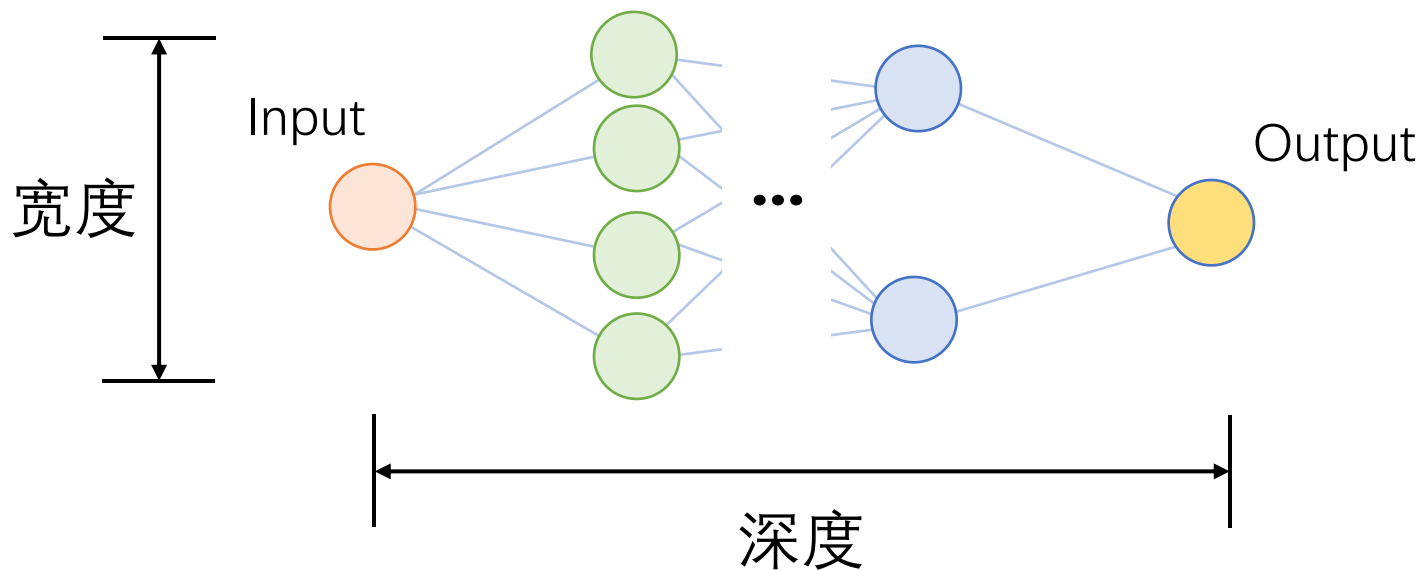
其中  $w^T x$  是输入特征与权重的加权求和， $b$  为偏置项。随后将结果输入激活函数得到输出，从而引入非线性表达能力。然而，单层感知机只能学习线性决策边界，因此只能解决线性可分问题，无法处理 XOR 等非线性问题。

# 1.3 深度神经网络基础

## • 多层感知机

单个感知机先完成仿射变换，再通过激活函数输出结果：

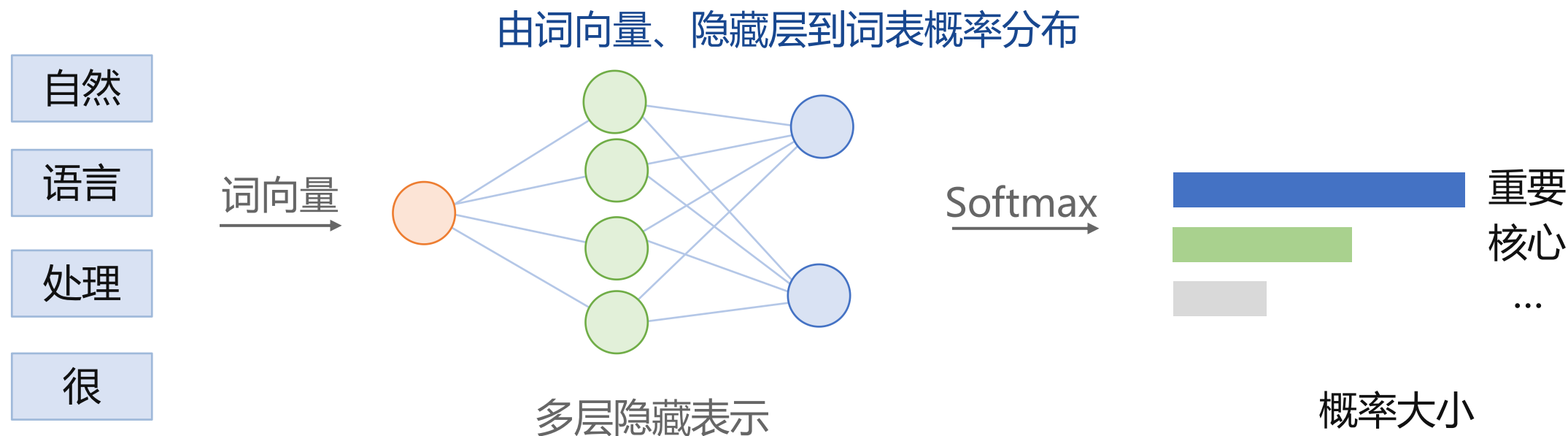
- 多个神经元组成一层，多个层构成函数复合。
- 深度描述层数，宽度描述每层神经元数量。
- 隐藏层将离散输入转为连续的上下文表示。



# 1.3 深度神经网络基础

- 神经语言模型

**语言模型示例：**给定“自然 语言 处理 很”，模型通过Softmax 预测下一个词，例如“重要”。

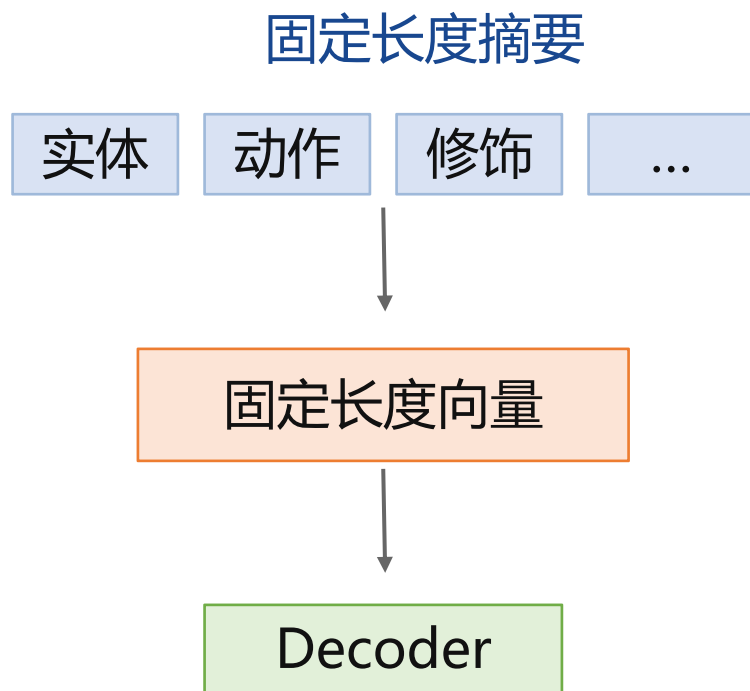


**隐藏层的作用：将离散词序列映射为可用于预测的连续上下文表示。**

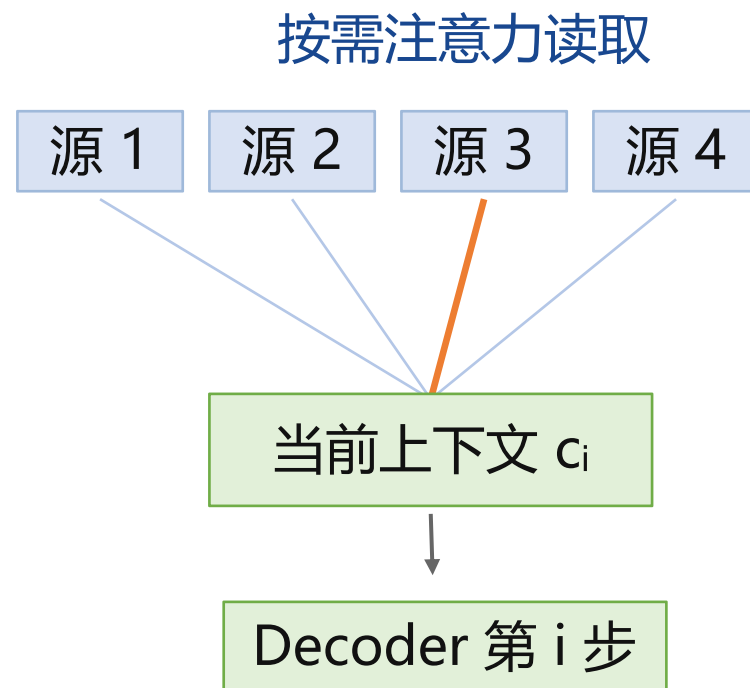
# 1.3 神经网络基础

- 固定长度向量与注意力机制

- 早期 Encoder/Decoder 把整个源序列压缩为一个固定长度向量。
- 长句中的实体、动作和修饰关系很难被单一向量完整保留。
- 注意力机制允许解码器在每一步读取与当前预测最相关的源端信息。



信息统一压缩，长序列容易丢失细节

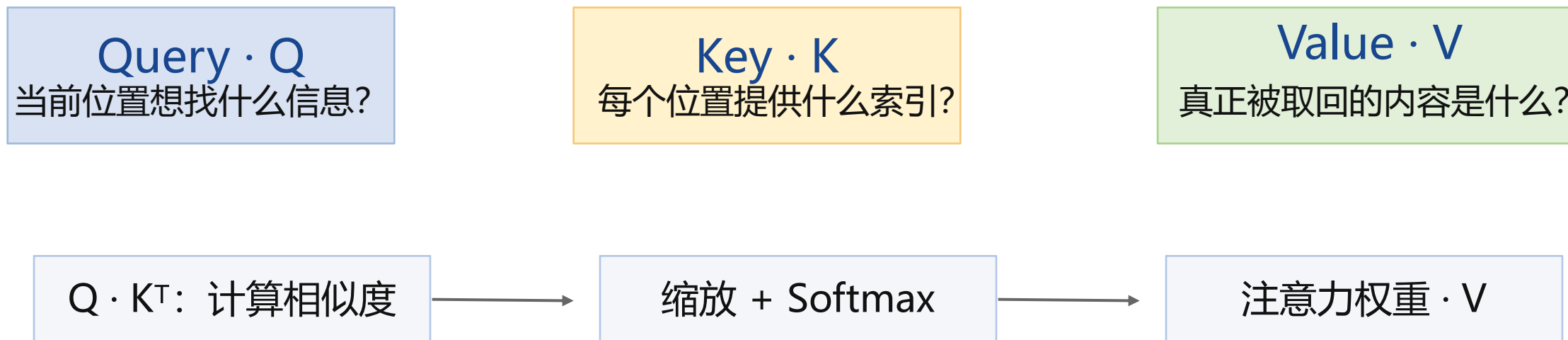


当前词只聚合与其最相关的源端位置

# 1.3 深度神经网络基础

- QKV: 注意力机制的检索视角

- Query 表示当前位置想要检索的信息; Key 提供每个位置的索引; Value 是被取回的内容。
- 模型根据 Query 与 Key 的相似度得到注意力权重, 再对 Value 加权求和。



Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

注意力权重只决定 “从哪里取信息” ; 最终表示是 Value 的加权和。

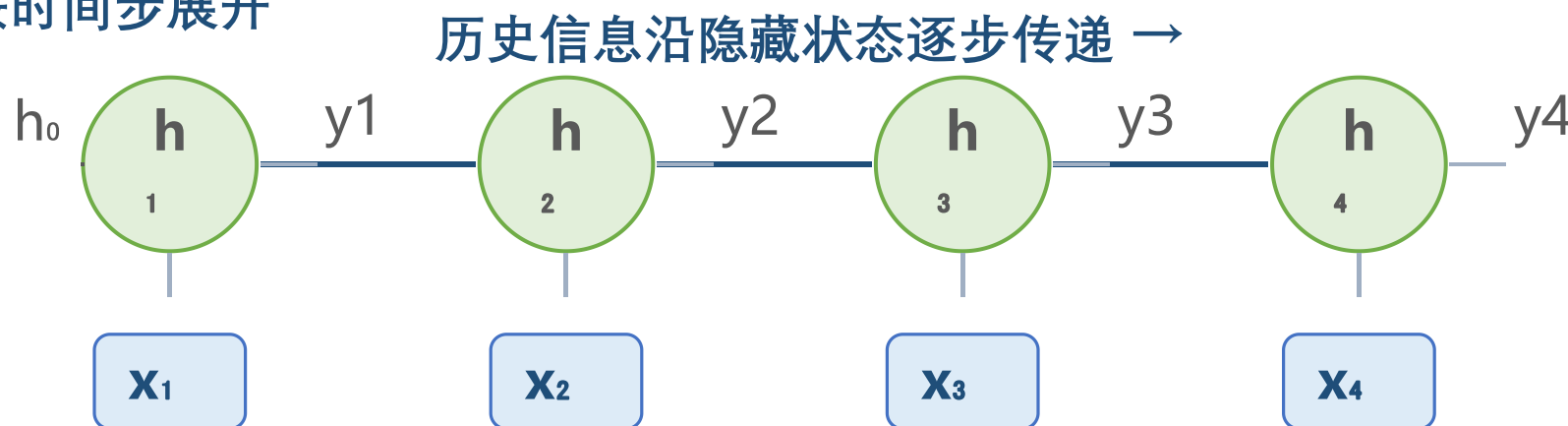
# 1.3 深度神经网络基础

## RNN: 用隐藏状态递归处理序列

序列数据具有顺序依赖：处理当前位置时，需要保留前面位置的信息。RNN通过隐藏状态  $h_t$  递归传递历史信息。

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

按时间步展开



RNN 的核心

当前输入  $x_t$   
+  
历史状态  $h_{t-1}$   
↓  
新的状态  $h_t$

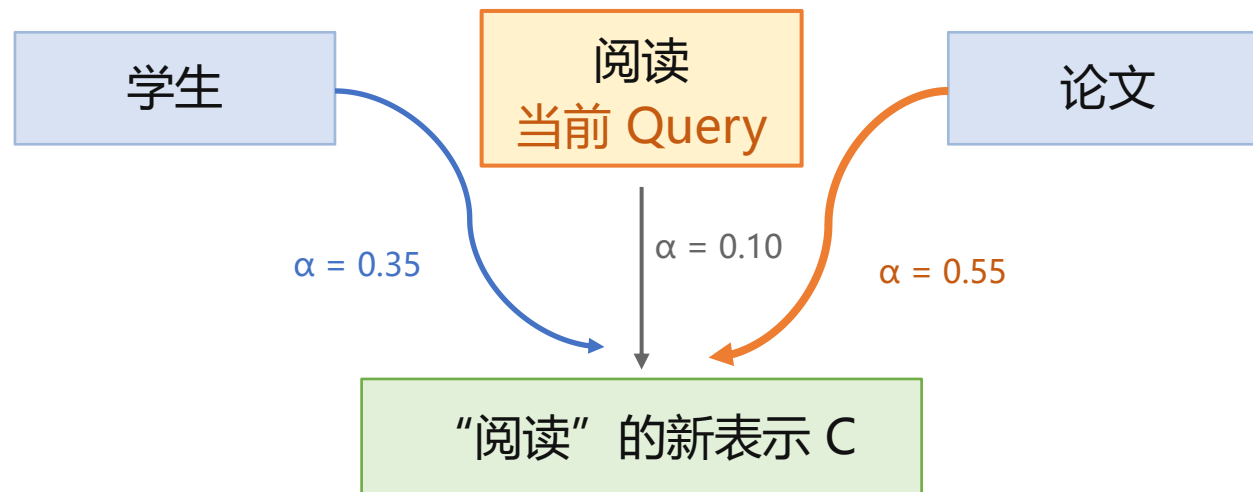
特点：顺序处理、状态递归更新；因此天然适合建模序列，但长距离信息需要逐步传递。

# 1.3 深度神经网络基础

## ● 自注意力机制

- 输入序列  $H$  经过线性变换生成  $H^Q$ 、 $H^K$ 、 $H^V$ 。 示例：更新“阅读”位置的表示
- 每个位置都可与序列中其他位置直接交互。
- 输出  $C$  是融合全局上下文后的新表示。

$$\text{Self-Attention}(H) = \text{Softmax}\left(\frac{H^Q(H^K)^T}{\sqrt{d}}\right)H^V$$



**与 RNN、CNN 相比：**

任意两个位置之间的信息传递距离更短。

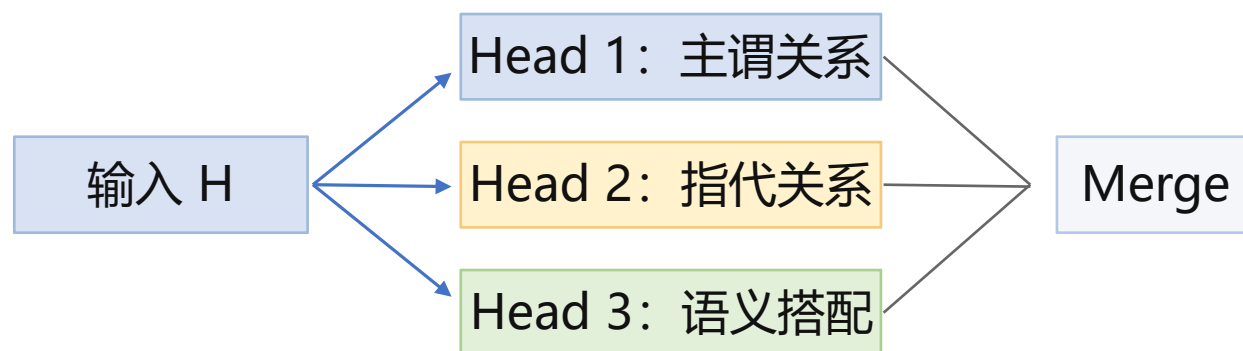
“阅读” 同时融合主语、宾语与自身信息。

# 1.3 深度神经网络基础

## ● Multi-head Attention 与位置编码

- 多个 head 在不同的低维子空间独立执行注意力。
- 不同 head 可学习不同的位置关系、语法线索或语义关联。

同一输入 H 投影到多个注意力子空间

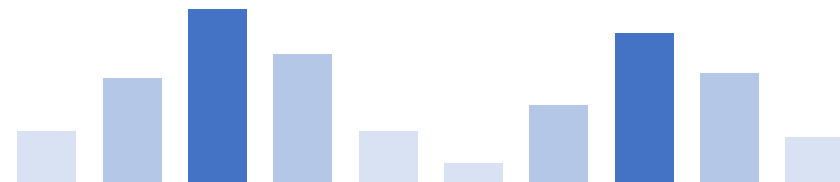


位置编码：让模型区分 token 的先后

- 普通注意力与 FFN 本身不感知词序，
- 因此 Transformer 将词向量与位置表示相加：

$$e_j = x_j + PE(j)$$

原始 Transformer：正弦 / 余弦绝对位置编码



每个位置获得唯一的连续向量表示。

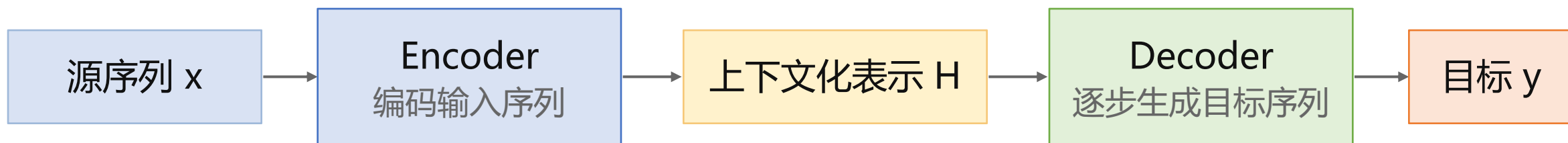
多头注意力提供多种关系视角；位置编码保留序列的先后顺序。

# 1.3 神经网络基础

- Encoder/Decoder 结构

- Encoder 将源端序列  $x$  映射为中间表示  $H$ :  $H = \text{Encode}(x)$ 。
- Decoder 根据  $H$  与已生成前缀, 逐步预测目标端序列  $y$ :  $y = \text{Decode}(H)$ 。

先理解输入, 再按条件生成输出



典型序列到序列任务

机器翻译

文本摘要

条件问答

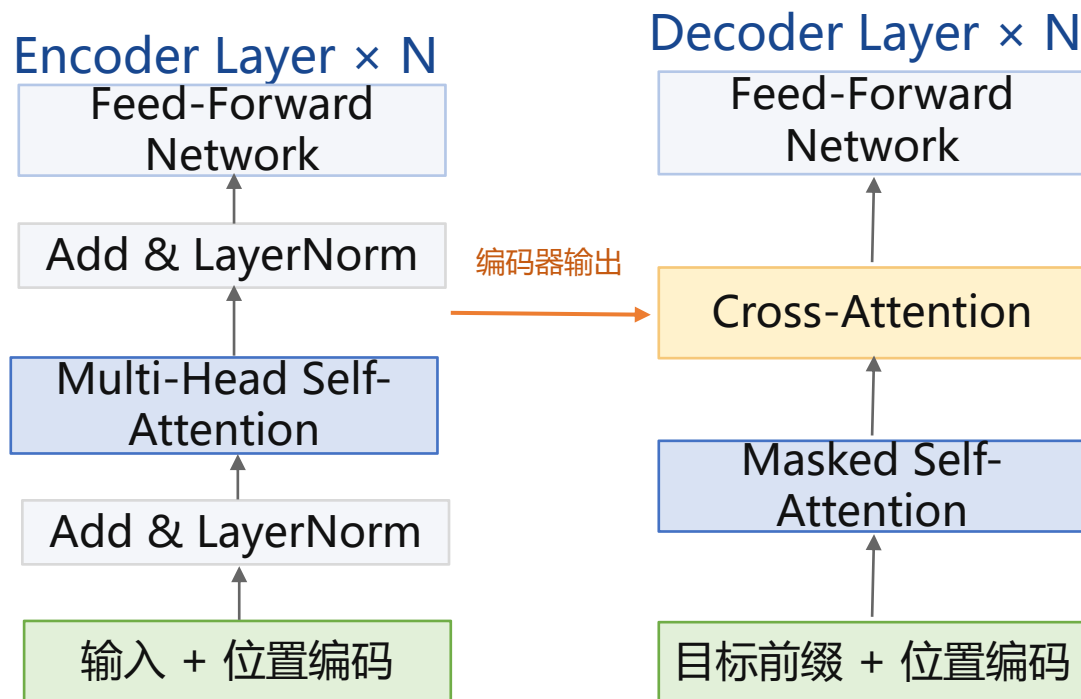
预测时, 系统在候选目标序列中寻找条件概率最大的结果。

# 1.3 深度神经网络基础

- Transformer 的核心组件
  - Encoder 由 Self-Attention 子层和前馈网络 (FFN) 子层构成。
  - 每个子层之后配合残差连接 (Add) 和层归一化 (LayerNorm) 。
  - Decoder 在此基础上增加 Masked Self-Attention 与 Cross-Attention。
  - Masked 机制保证当前位置不能看到未来 token。

Self-Attention → Add & Norm → FFN → Add & Norm  
Decoder self-attention 建模目标端前缀; Cross-attention 读取源端表示。

Transformer 的 Encoder / Decoder 结构



# 1.3 深度神经网络基础

- Transformer 的三类常见架构
  - Transformer 可以按是否保留 Encoder 与 Decoder 分为三类。
  - 核心差异在 token 可见的信息范围，以及输入和输出的组织方式。

是否双向理解输入、是否按前缀生成、是否先读入再生成，决定了架构的适用任务。

## Encoder-only

输入 token  $\leftrightarrow$  双向上下文

擅长：理解与编码  
用途：分类、抽取、检索

## Decoder-only

已生成前缀  $\rightarrow$  下一个 token

擅长：自回归语言建模  
用途：对话、续写、代码生成

## Encoder-Decoder

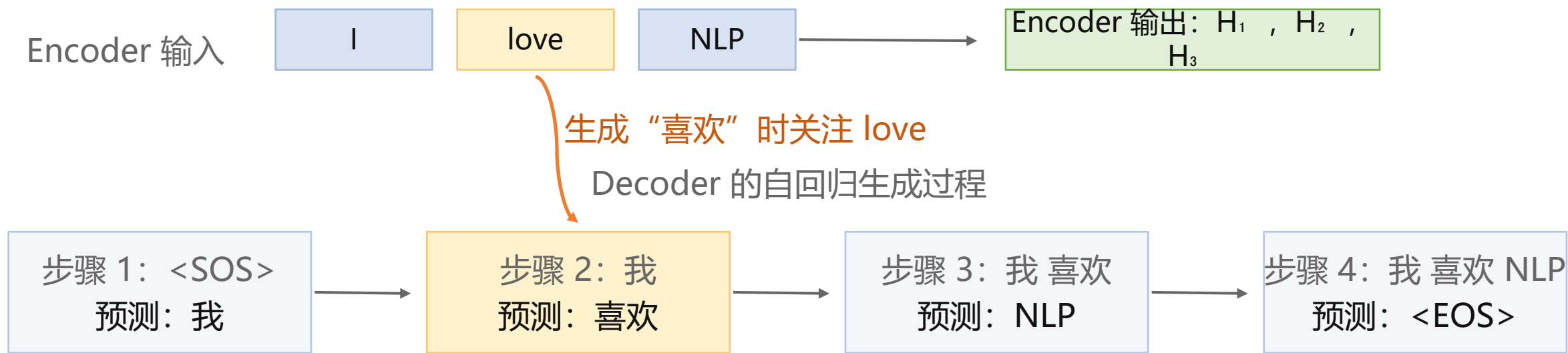
输入序列  $\rightarrow$  目标序列

擅长：条件序列到序列  
用途：翻译、摘要、问答

# 1.3 深度神经网络基础

- 机器翻译中的 Encoder/Decoder 示例

源句 “I love NLP” 经 Encoder 形成上下文表示；Decoder 从起始符号开始逐步生成中文译文。



**Decoder Self-Attention:** 读取已生成前缀。

**Cross-Attention:** 读取最相关的源端表示。

每一步均结合目标端历史和源端语义，经 Softmax 输出下一个词的概率分布。

# 1.3 深度神经网络基础

- 小结

- **1. MLP 学习逐层表示变换**

隐藏层将离散输入映射为连续表示，并在语言模型中用于预测下一个词。

---

- **2. Attention 学习动态信息选择**

QKV 通过相似度得到权重，再对 Value 加权聚合，形成融合上下文的新表示。

---

- **3. Encoder/Decoder 组织输入到输出**

Transformer 以自注意力、交叉注意力、残差连接和归一化构成现代序列建模基础。

# 第一章

## 第1.4讲标题

肖桐 杨曦涵 李天园 刘宏宇

xiaotong@mail.neu.edu.cn

东北大学 计算机科学与工程学院 人工智能系 自然语言处理实验室



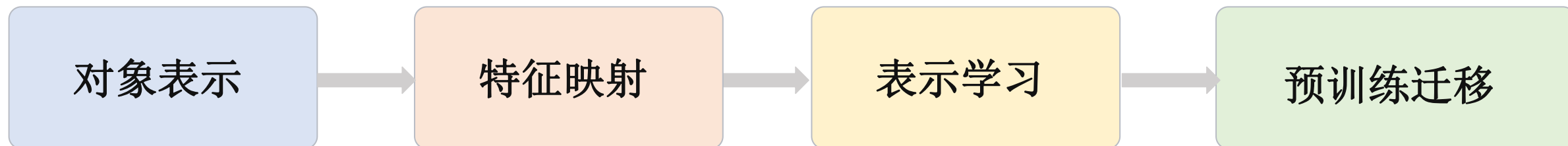
東北大學  
Northeastern University



## 1.4 深度神经网络基础

- 模型只能处理数值向量或张量，现实对象必须先被数值化。
- 好的表示保留任务相关信息，使相似对象在表示空间中具有相近结构。
- 本章主线：对象表示 → 表示学习 → 预训练迁移。

表示不是数据的“副本”，而是连接现实对象与学习算法的接口。



# 1.4 表示学习机制

## One hot 编码

- One-hot 能准确区分离散对象，却没有表达对象之间的相似性。

$$\mathbf{x}_{\text{猫}} = [1, 0, 0]^T, \quad \mathbf{x}_{\text{狗}} = [0, 1, 0]^T, \quad \mathbf{x}_{\text{汽车}} = [0, 0, 1]^T$$

- 任意两个不同对象的点积都为 0，欧氏距离都为  $\sqrt{2}$ 。
- 当对象数量增大时，向量维度和参数矩阵随之增长，表示高度稀疏。
- 不同对象之间不能共享统计信息：从“猫”学到的规律不会自然帮助模型处理“狗”。

局限性归纳：高维、稀疏、无相似性、难以共享统计信息。

# 1.4 表示学习机制

## 分布式表示

- One-hot 能准确区分离散对象，却没有表达对象之间的相似性。

$$\mathbf{e}_s = \mathbf{E}^T \mathbf{x}_s \in \mathbb{R}^d, \quad d \ll |V|$$

- 低维稠密向量替代高维稀疏编码。
- 距离与方向承载对象之间的相似性和关系。
- 共享特征使一个对象上学到的规律能够迁移到相似对象。

“分布式”不是概率分布，而是信息分散在多个共享维度中。

# 1.4 表示学习机制

- 特征映射回答：模型应该用哪些数值描述输入，才能保留与预测目标有关的信息？

$$s \xrightarrow{\phi} \mathbf{z} = \phi(s) \in \mathbb{R}^d$$

原始对象  
像素 / 词序列

数值化  
向量或张量

信息筛选  
突出任务相关性

特征空间  
更易泛化

- 好的映射同时完成：可计算、抑制噪声、提升有限数据上的泛化。
- 映射可以由人工规则给定，也可以从数据估计，或与预测模型一起端到端学习。

“更适合任务的空间”不是预先存在的，而是通过表示设计与学习过程逐渐形成。

## 1.4 表示学习机制

- 词袋模型完成“文档 → 向量”，但把所有词的出现次数同等看待。
- TF-IDF 通过逆文档频率降低常见词的影响，提高区分度高的词的权重。

$$\text{tfidf}(a, d, D) = \text{tf}(a, d) \cdot \text{idf}(a, D), \quad \text{idf}(a, D) = \log \frac{|D|}{\text{df}(a, D)}$$

### “模型”

在 1000 篇文档中出现 900 篇，IDF 较低。即使词频相同，最终权重也较小。

### “过拟合”

只在 20 篇文档中出现，IDF 较高，因而获得更强的区分度。

特征映射不仅把输入变成数字，也把“哪些信息可能更有用”编码进表示。

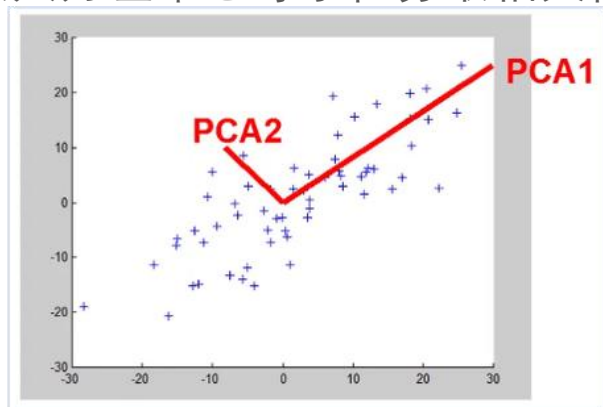
# 1.4 表示学习机制

序列数据具有顺序依赖：处理当前位置时，需要保留前面位置的信息。RNN通过隐藏状态  $h_t$  递归传递历史信息。

## 固定映射：PCA

$$\mathbf{N} = \mathbf{M}\mathbf{C}, \quad r < d$$

- 选择数据方差较大的主要方向。
- 在减少维度的同时保留总体结构。
- 最大方差未必等于任务最相关信息。

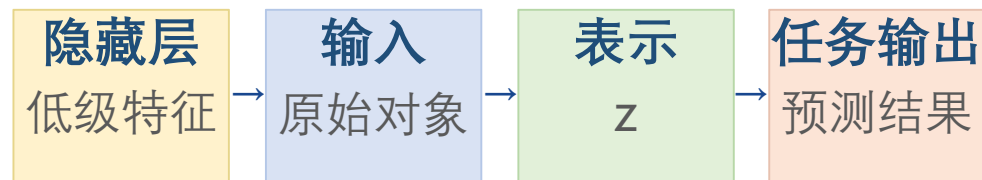


二维数据投影到一维主方向

## 可学习映射：神经网络

$$\mathbf{z} = \phi_{\theta}(s)$$

- 映射本身由参数  $\theta$  表示，并可依据任务损失端到端更新。



**层级组合：**边缘 / 词形  $\rightarrow$  纹理 / 局部语义  $\rightarrow$  形状 / 句意  $\rightarrow$  任务决策。

固定映射强调“保留总体结构”；可学习映射强调“让表示服务于任务”。

## 1.4 表示学习机制

大模型时代的表示学习机制

# 1.4 表示学习机制

- 预训练先利用规模更大、来源更广的数据学习一个通用参数起点。

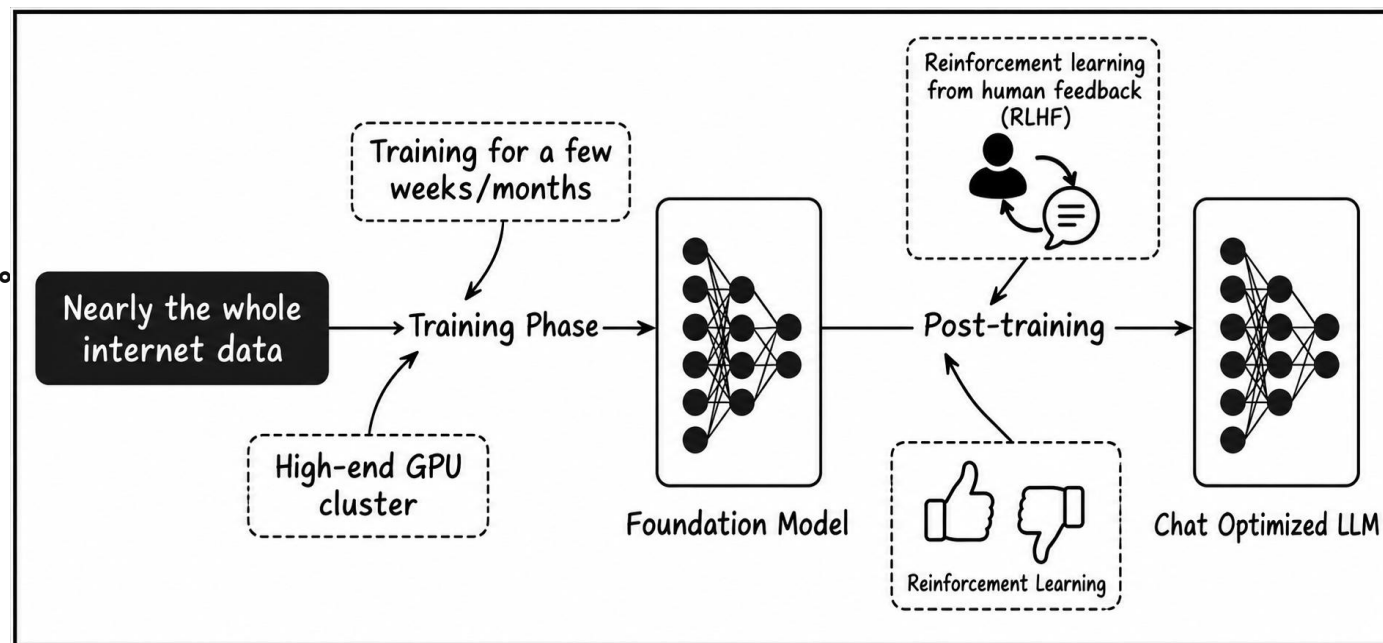
$$\theta_{pre} = \arg \min_{\theta} \mathcal{L}_{pre}(\theta; \mathcal{D}_{pre})$$

- 目标任务从  $\theta_{pre}$  出发，而不是从随机参数开始。
- 收益：提高数据利用效率、改善优化起点、复用一次大规模训练成本。

大规模预训练数据

通用表示与参数

目标任务微调适配



预训练、特征提取与微调的知识复用流程

预训练是一种通用迁移机制，可用于文本、图像、语音、图与分子数据。

核心范式：先学习可复用结构，再适配目标任务。

# 1.4 表示学习机制

- 预训练编码器可以通过不同方式服务多个下游任务：

## 特征提取

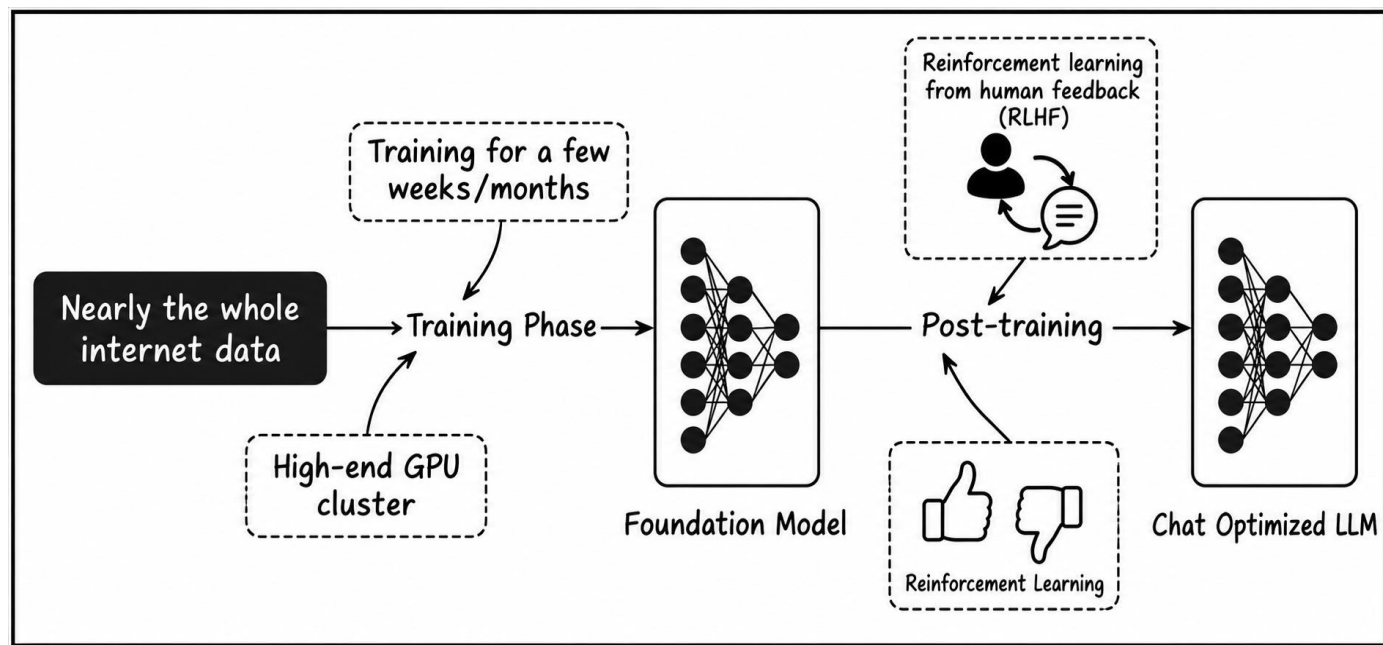
冻结预训练参数，只训练小型任务模型。

## Fine-tuning

以预训练参数为起点，更新部分或全部参数。

## Prompting

通过任务描述和示例引导模型完成任务。



预训练、特征提取与微调的知识复用流程

目标任务决定哪些表示真正有用，最终需要通过泛化性能、评估与对齐检验。

对象表示 → 特征学习 → 预训练 → 迁移适配 → 任务评估

# 1.4 表示学习机制

- 小结

- **1. 对象表示**

One hot 编码, 分布式表示

---

- **2. 表示学习**

词袋模型, TF-IDF和Word2Vec

---

- **3. 预训练迁移**

特征提取, Fine-tuning, Prompting